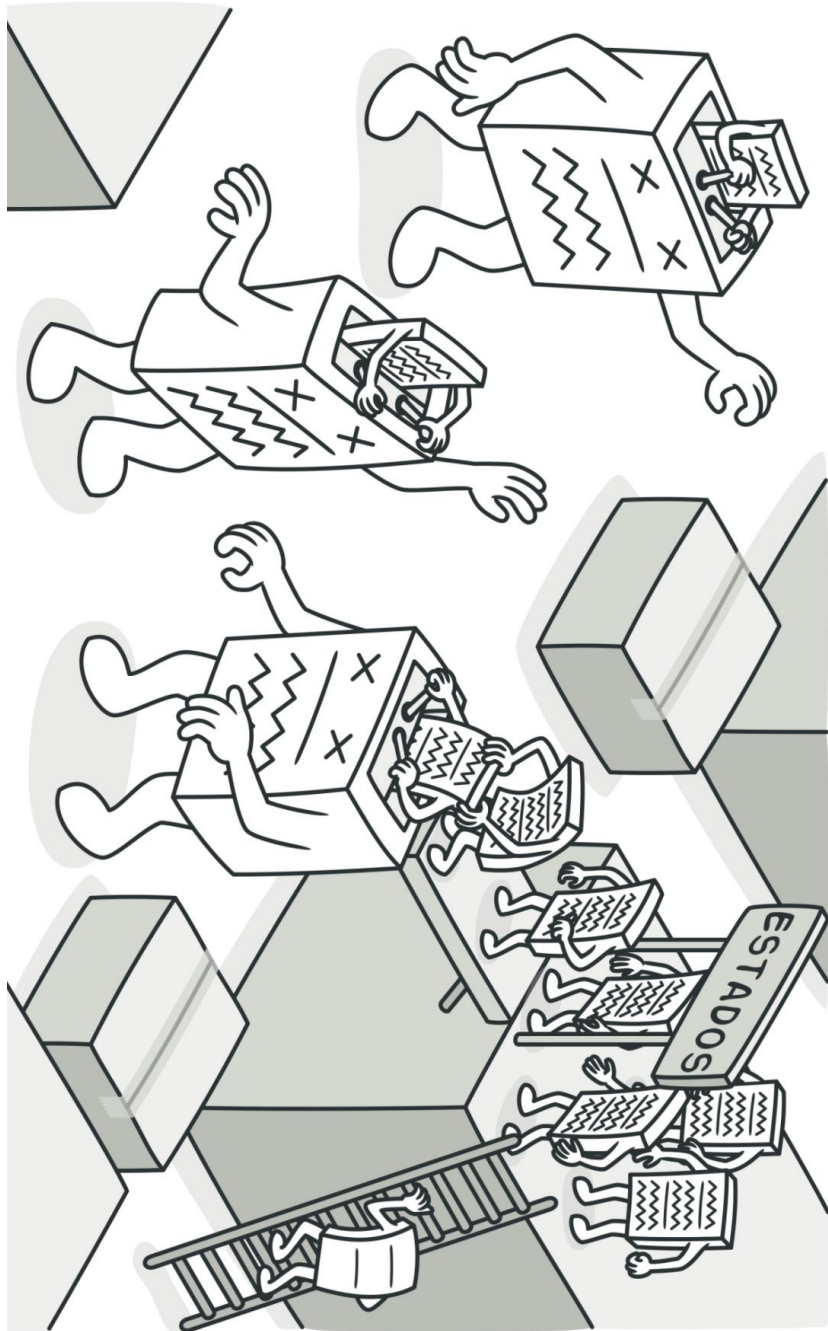
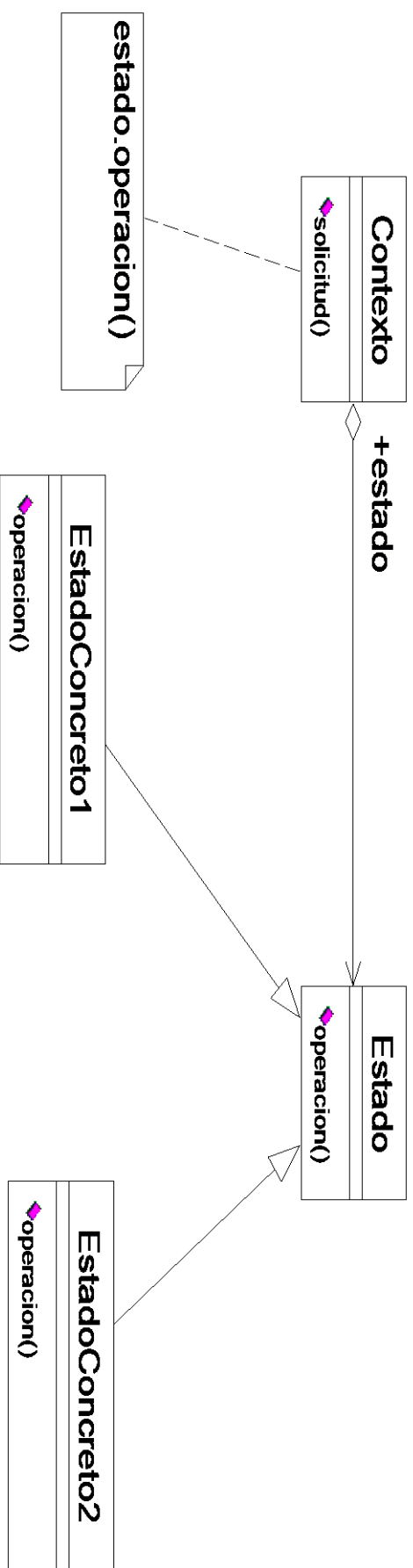


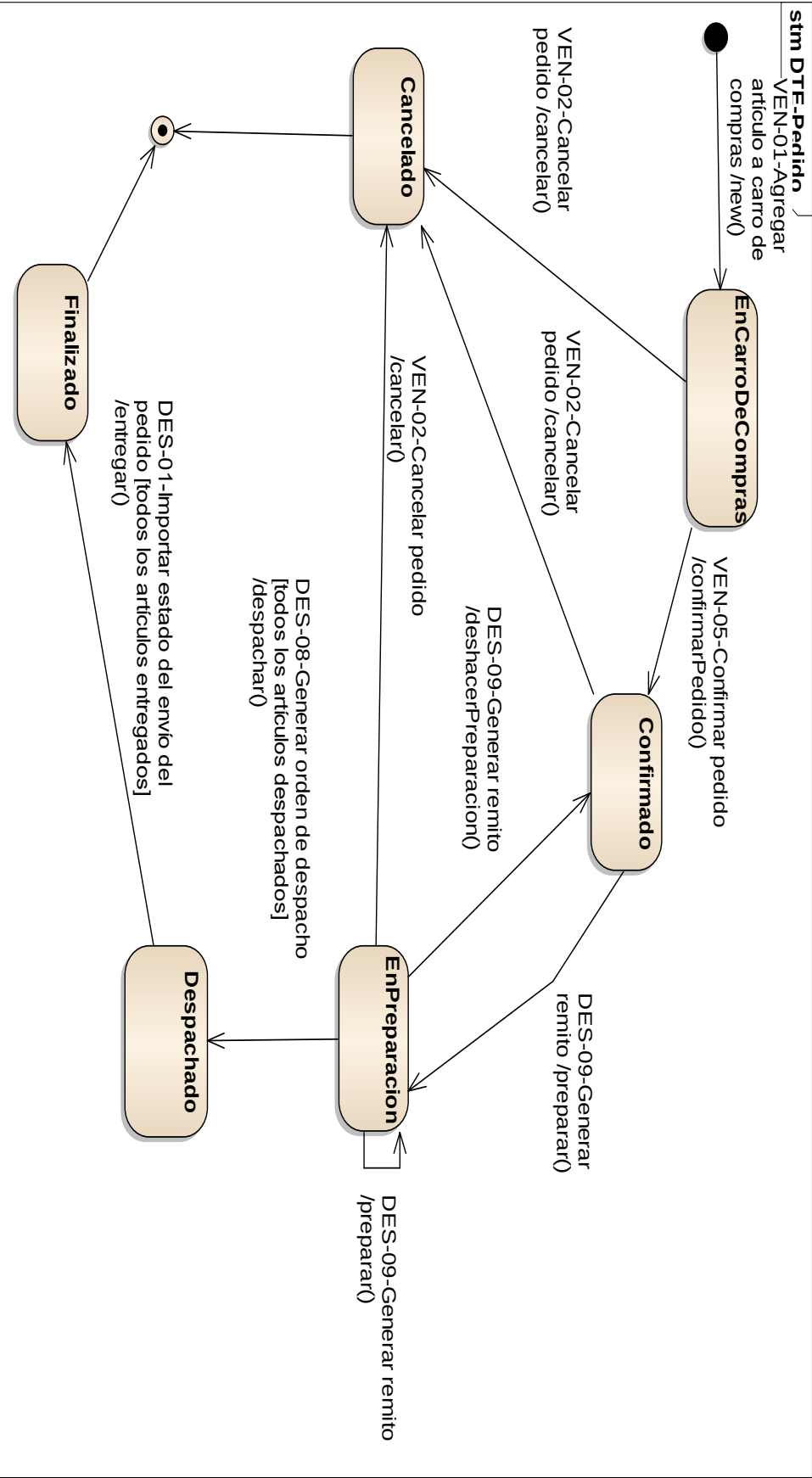
State



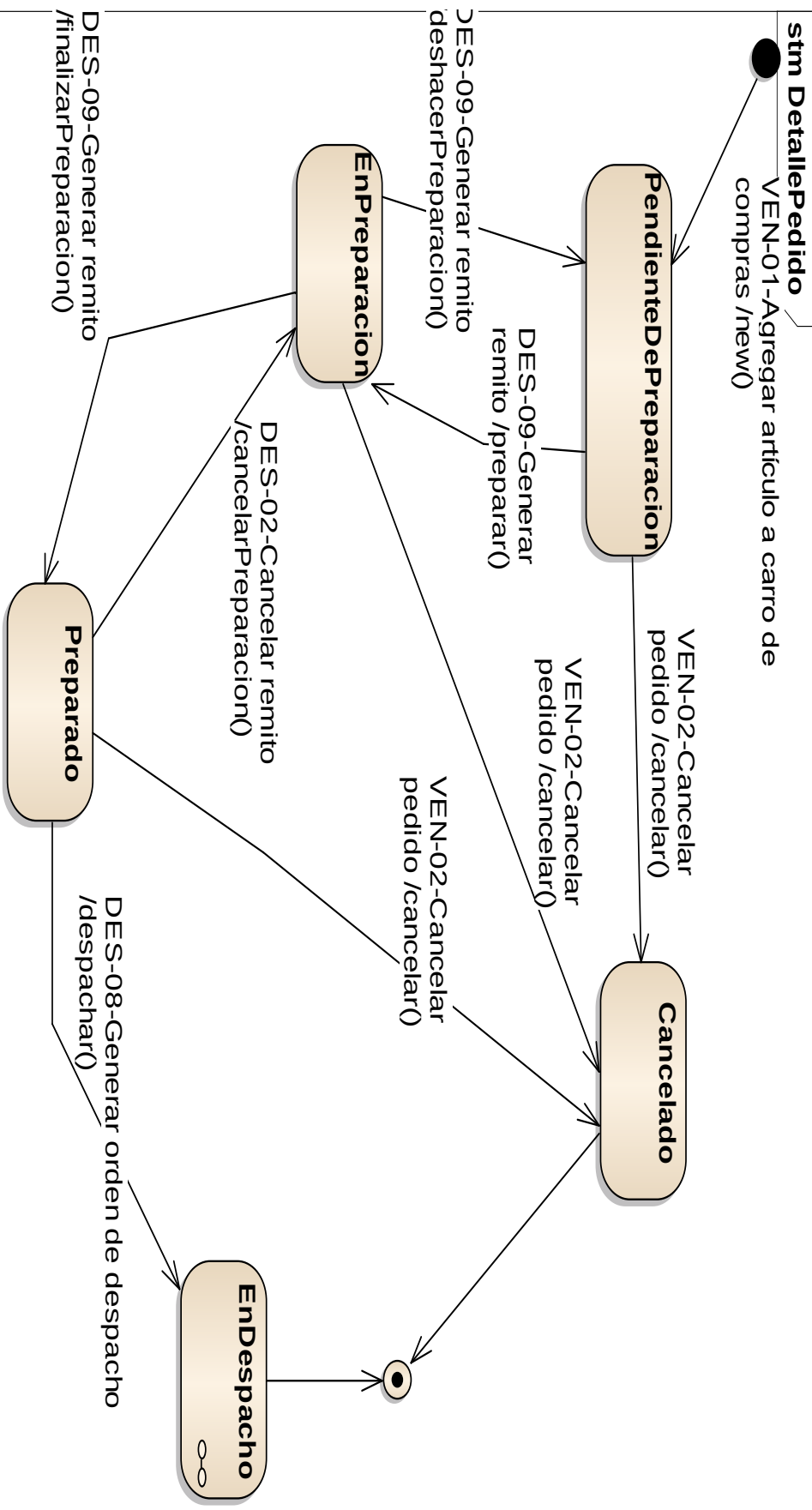
State (Estado)



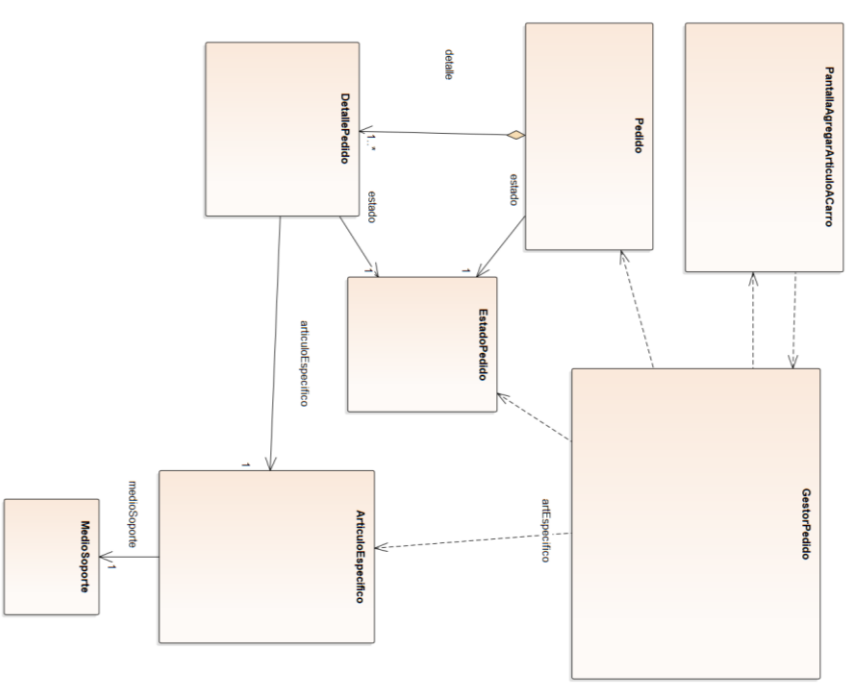
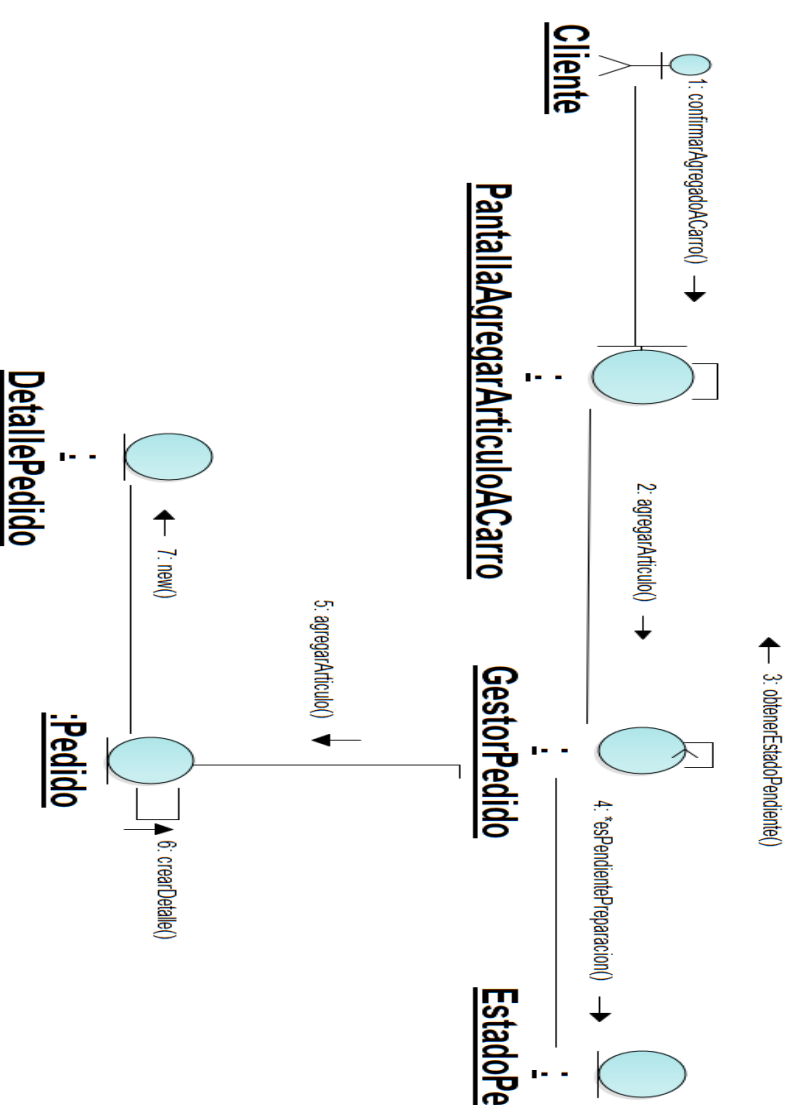
Máquina de Estado de la Clase Pedido



Máquina de Estado de la Clase DetallePedido



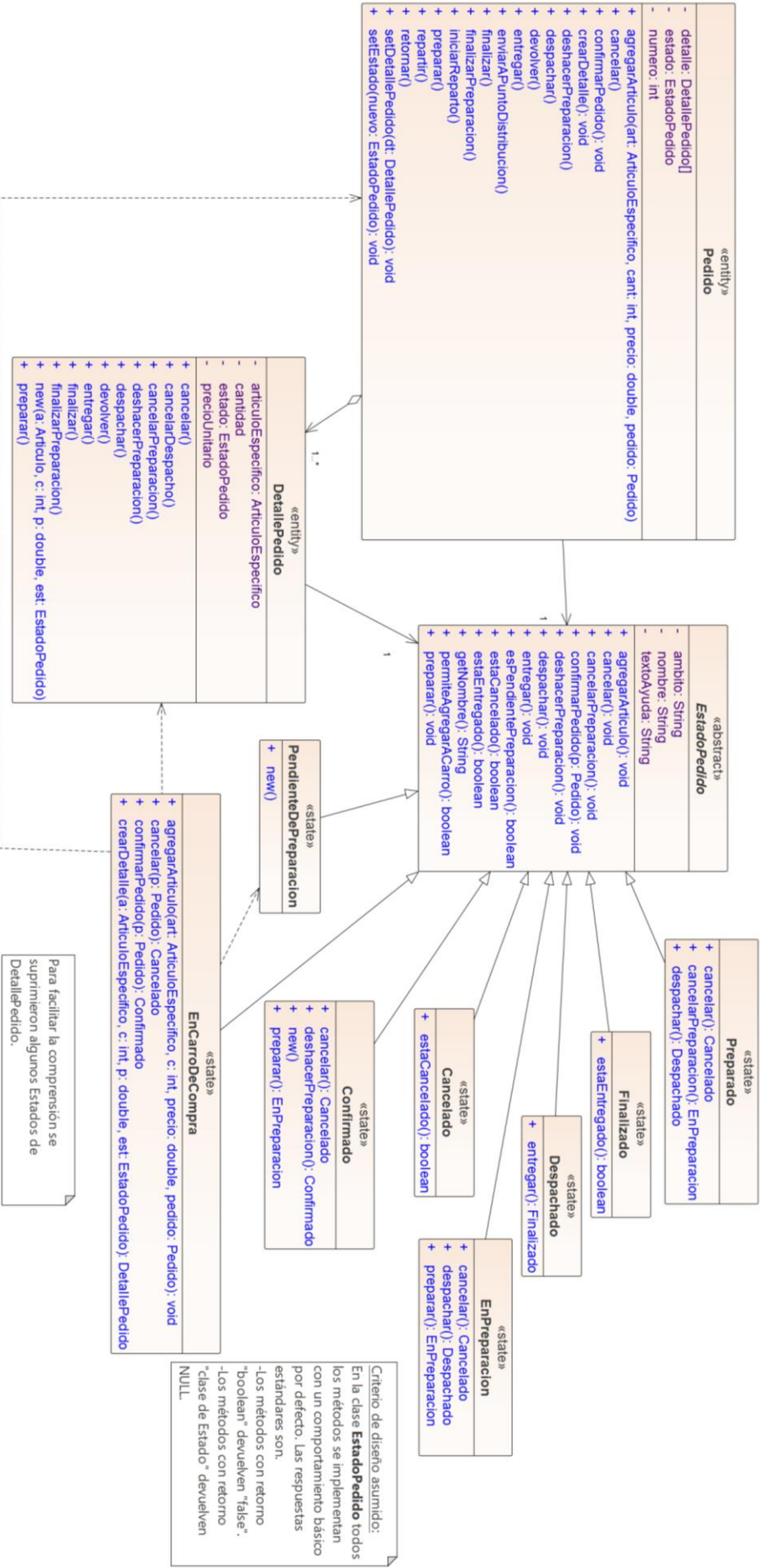
Vista parcial de la Realización de CU de Análisis del Caso de Uso Agregar Artículo al Carro de Compras



State: Aplicación

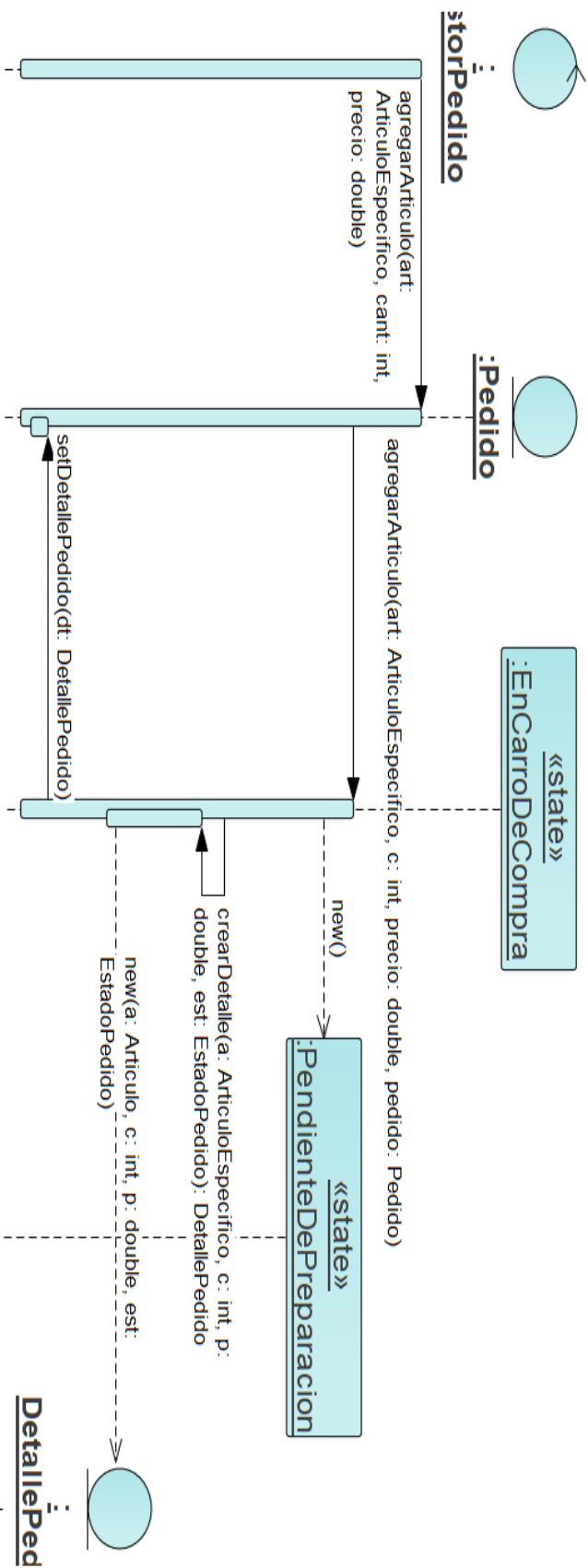


Estructura



State: Aplicación

⦿ Dinámica:

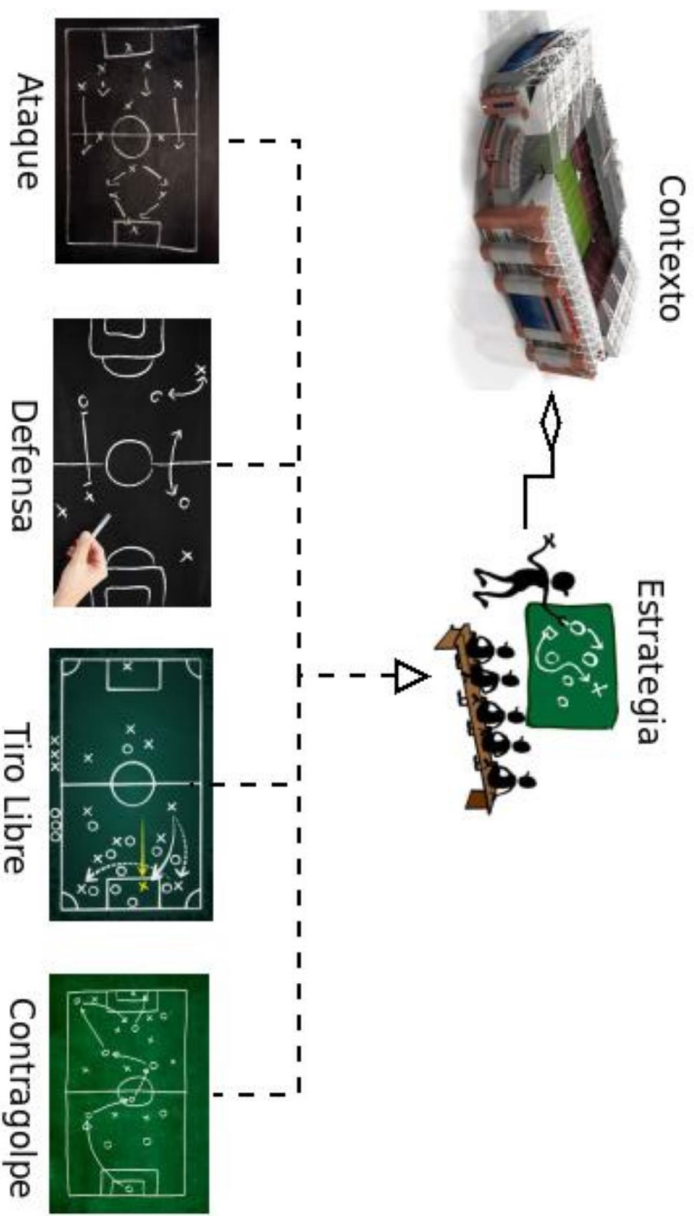


⦿ Ventajas:

- Se eliminan las sentencias CASE utilizando Polimorfismo.
- Permite crecimiento en la variación de la lógica de negocio en función de los cambios de estado.

Strategy

Strategy



Strategy/Policy (Estrategia)

◎ Propósito

- Define una familia de algoritmos, encapsula cada uno, y permite intercambiarlos. Permite variar los algoritmos de forma independiente a los clientes que los usan

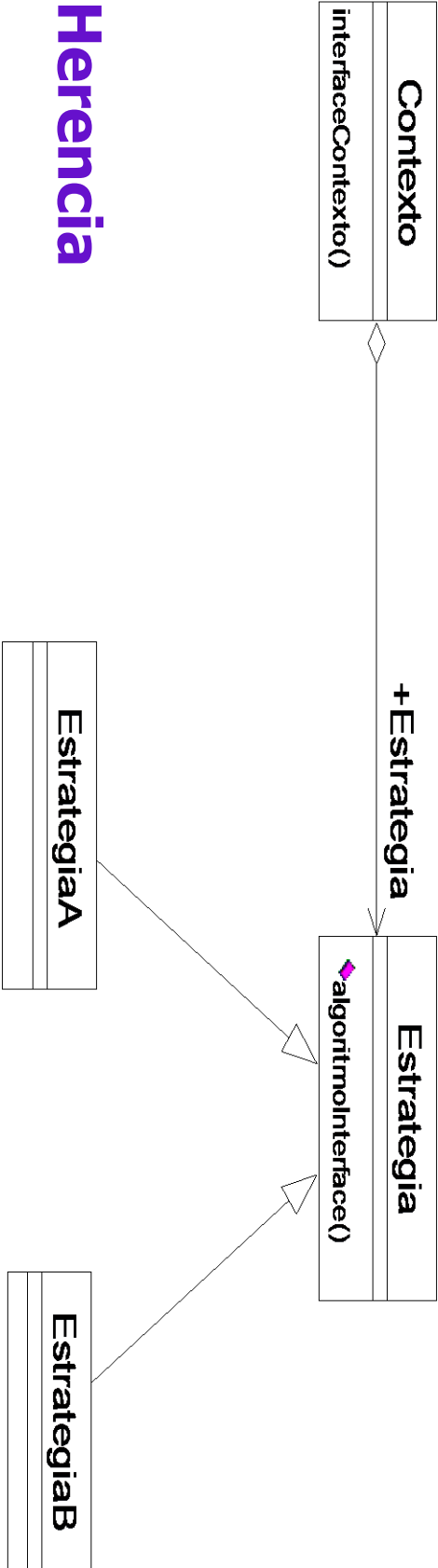
◎ Motivación

- Existen muchos algoritmos para justificación de texto, ¿debe implementarlo el cliente que lo necesita?

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Strategy

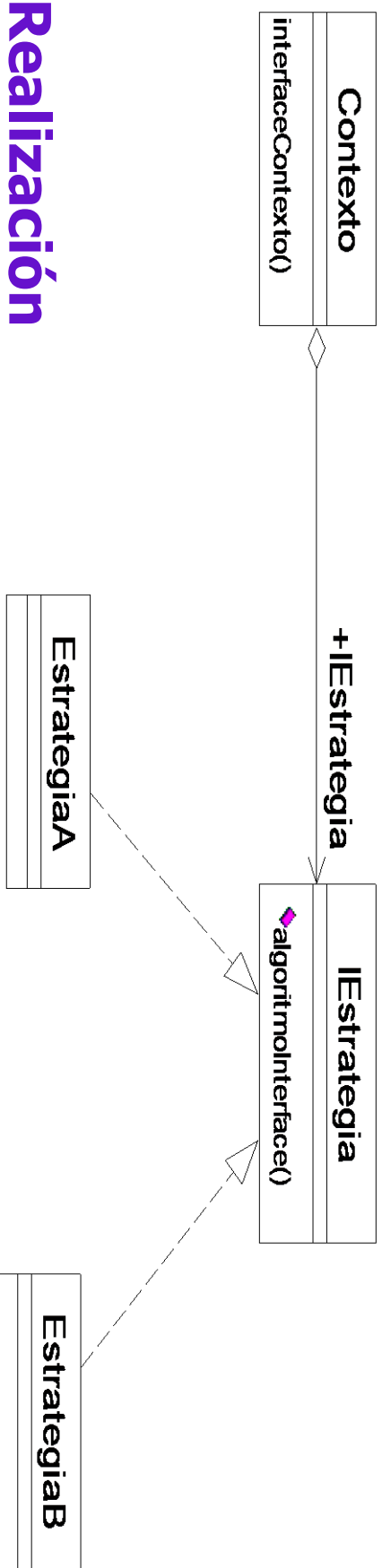
Estructura



Herencia

Strategy

Estructura



Realización

Flujo Básico

1- AC: selecciona la opción “Generar Diagramación de Campeonato”

2- **Sistema:** busca para el mes actual el campeonato que corresponde, que esté vigente y no tenga diagramación asociada y lo muestra, mostrando también el nombre del tipo de campeonato y el sistema de campeonato asociado.

3- **Sistema:** Busca los clubes inscriptos para el campeonato y calcula la cantidad de fechas y la cantidad de partidos a realizarse según el sistema de campeonato asociado, y las muestra.

4- **Sistema:** solicita que se indique la fecha de la primera jornada (fecha).

5- AC: Ingresa la fecha.

6- **Sistema:** Valida el sistema de campeonato seleccionado y **es todos contra todos**. Calcula las fechas de todas las jornadas sumando siete días a cada fecha y permite modificarlas.

7- AC: No desea modificar las fechas.

8- **Sistema:** Solicita que se ingrese la hora en que se jugará cada partido en las fechas.

9- AC: Ingresa la hora en que se jugará cada partido.

10- **Sistema:** El sistema de campeonatos seleccionado y **es todos contra todos**. *Para cada partido de cada fecha solicita* que se seleccione el club que será local y el club que será visitante, para la primera fecha y en función de eso determina las rondas restantes.

11- **Sistema:** Solicita se seleccionen los árbitros y el rol de cada uno (árbitro principal, juez de línea, cuarto árbitro)

12- AC: Selecciona los datos solicitados.

13- **Sistema:** Solicita se confirme la diagramación del campeonato.

14- AC: Confirma la diagramación del campeonato completa.

15- **Sistema:** valida que no exista más de un partido dónde se enfrenten los mismos equipos y no es así.

16- **Sistema:** Valida para cada partido que un mismo árbitro no esté seleccionado más de una vez para un partido (un árbitro puede cumplir un único rol en un partido) y que no se superpongan partidos para un árbitro y es así.

17- **Sistema:** El sistema de campeonato seleccionado y **es todos contra todos**. Registra la diagramación del campeonato, registrando para cada fecha los partidos a jugar con sus respectivas horas, clubes (local y visitante) y árbitros y asignando el estado “Diagramada” a cada fecha y partido.

Flujos Alternativos

A.1.. No hay ningún campeonato sin diagramación.

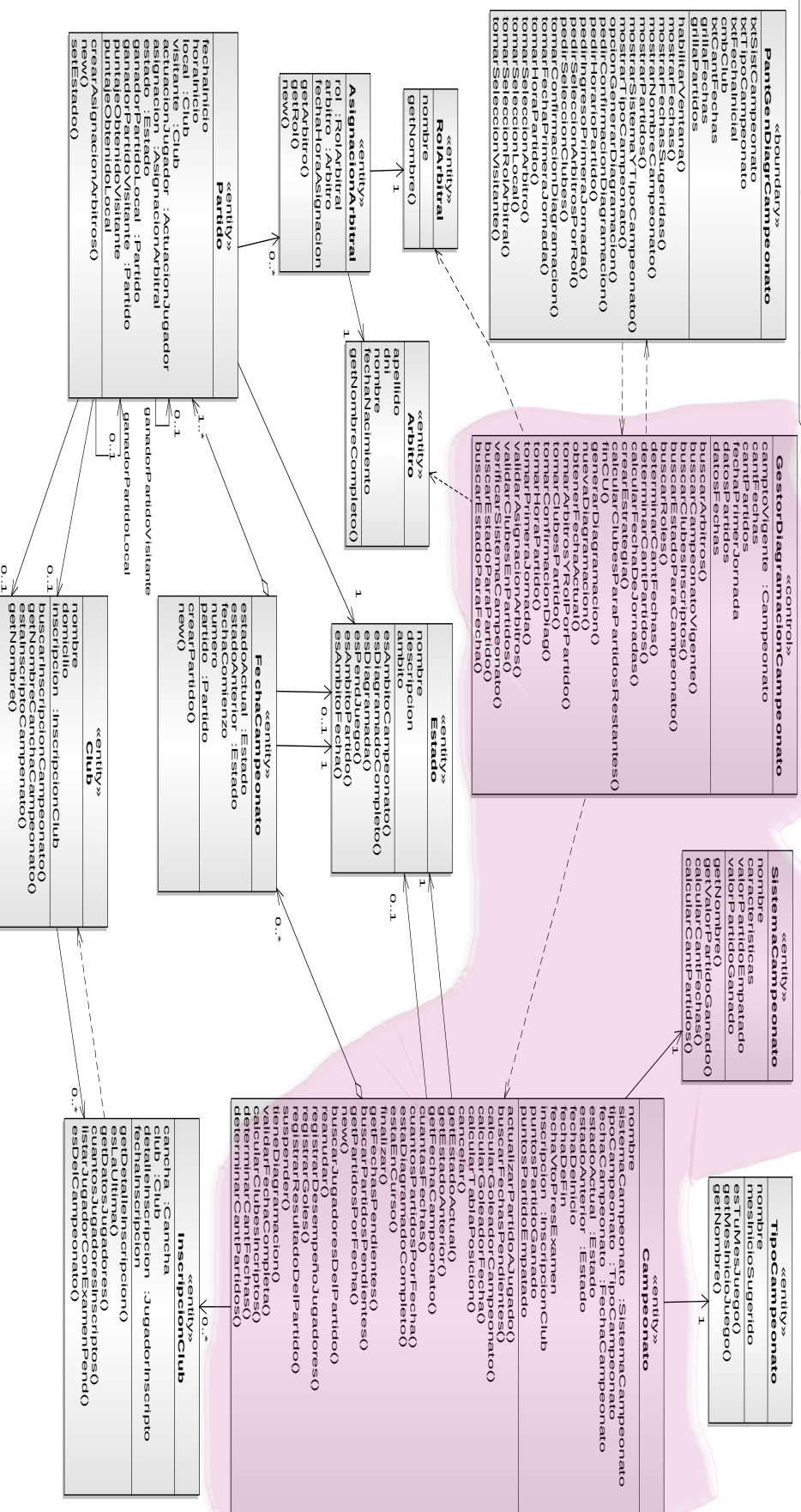
A.2.. El Sistema de campeonatos es **de eliminatorias**, debe calcular las fechas de todas las rondas sumando siete días entre cada ronda.

A.3.. Se modifican las fechas propuestas para cada jornada.

A.4.. El sistema de campeonatos seleccionado es **por eliminatorias**. Se debe definir clubes que jugarán, los árbitros y el rol de cada uno (árbitro principal, juez de línea, cuarto árbitro) para cada partido.

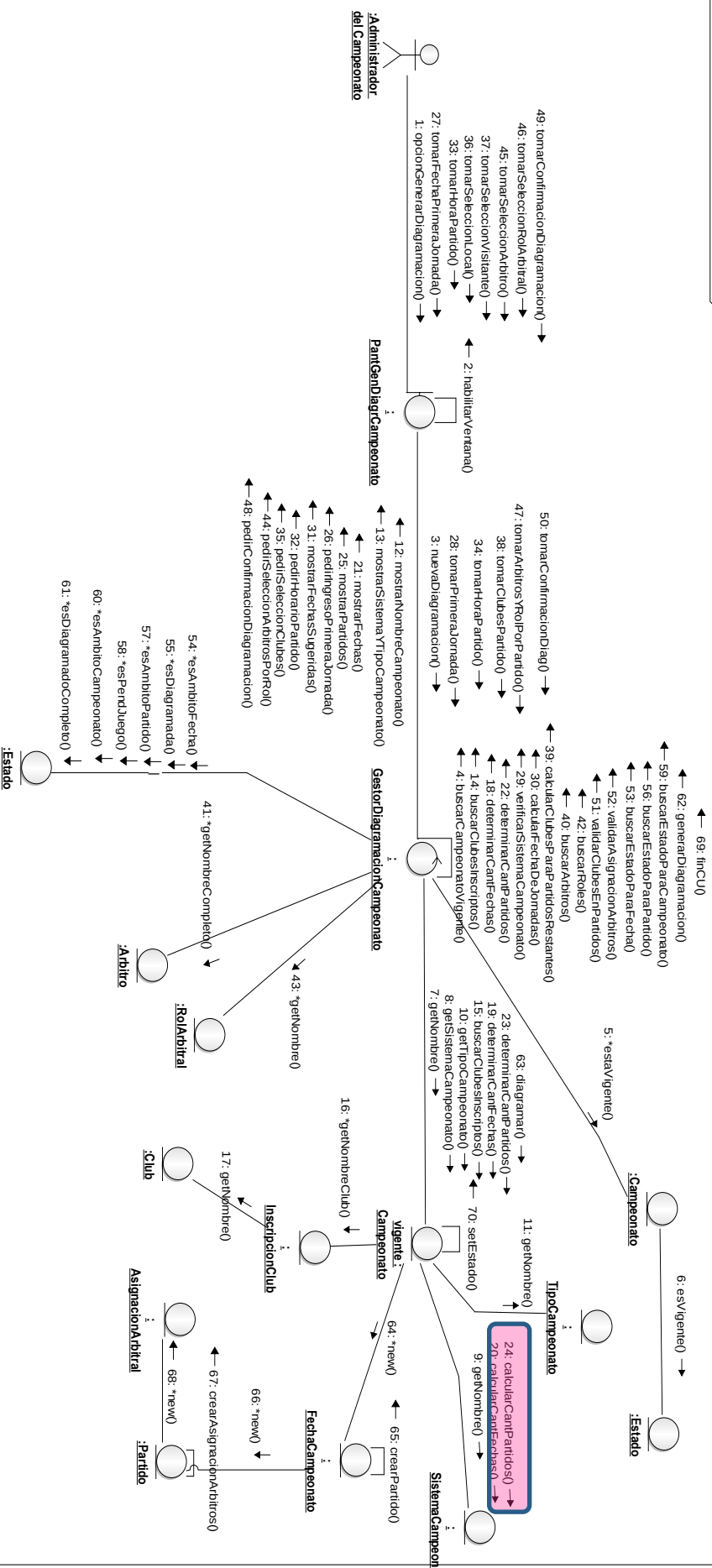
A.5 La diagramación del campeonato queda Incompleta.

Caso de Uso Generar Diagramación del Campeonato



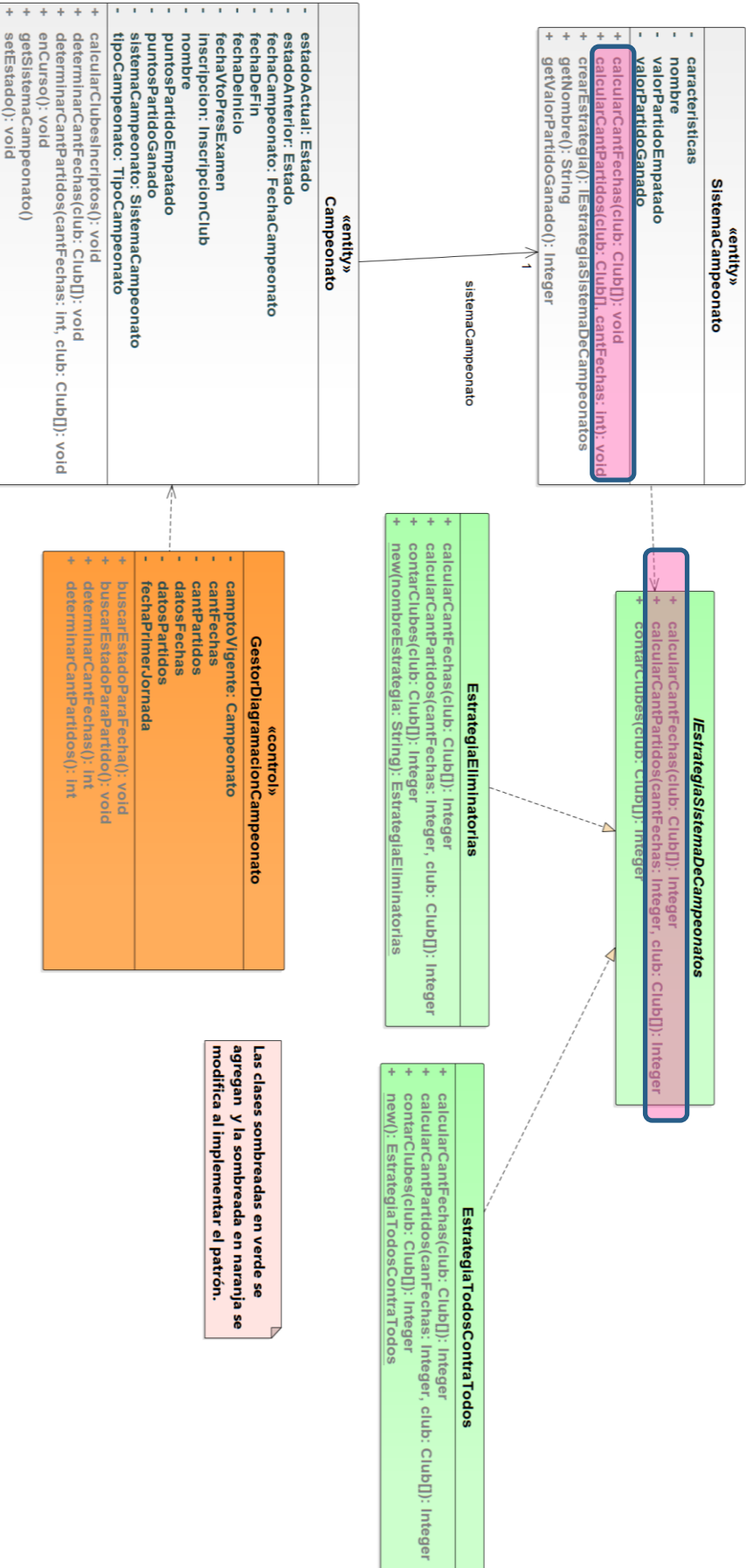
Caso Liga de Fútbol – Realización de CU de Análisis del

Caso de Uso Generar Diagramación del Campeonato



Strategy: Aplicación

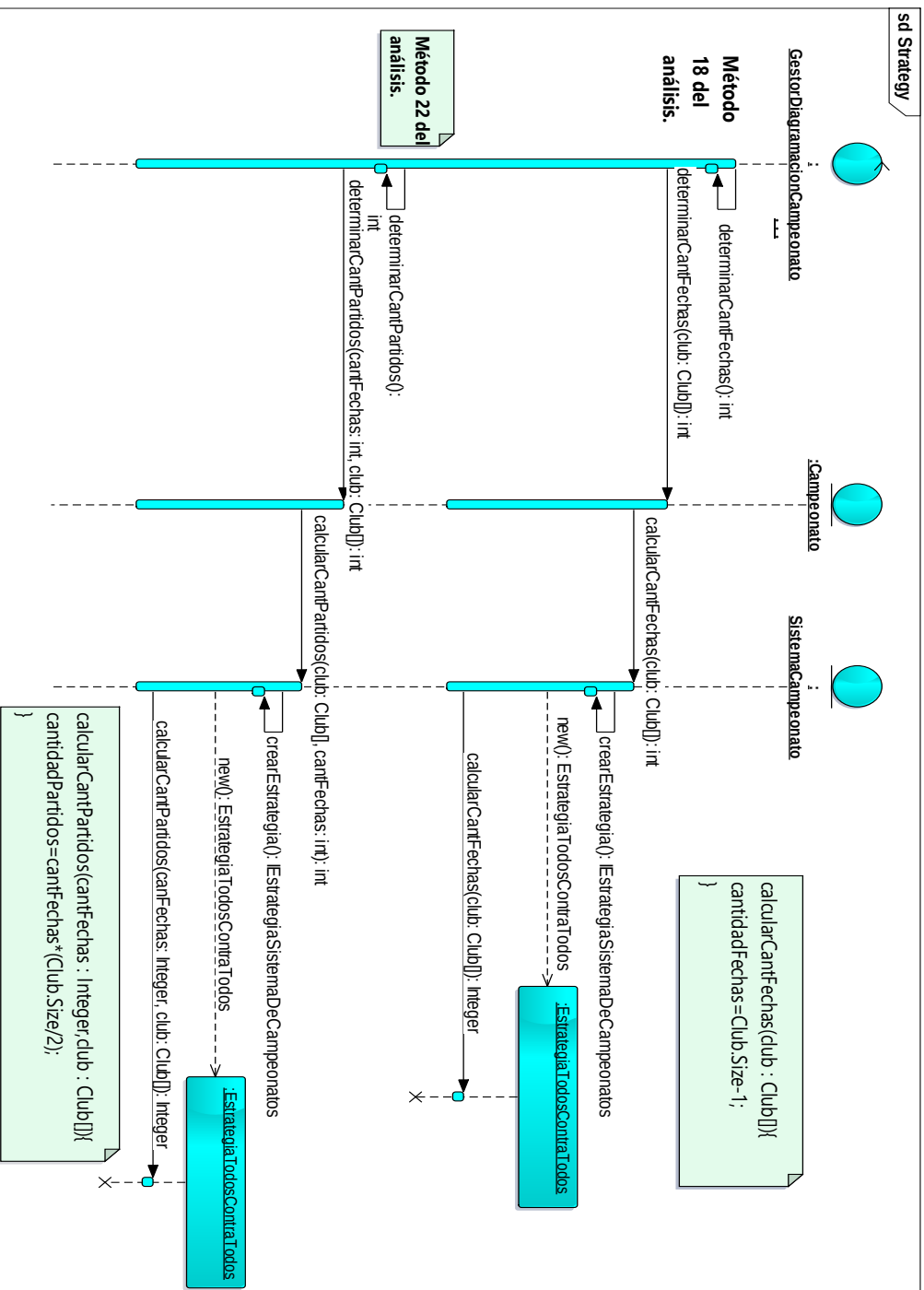
Estructura



Strategy:

Aplicación

•Dinámica



Strategy: Aplicación

⦿ Participantes

⦿ Estrategia [IEstrategiaSistemadeCampeonato]:

- Declara una interfaz común a todos los algoritmos permitidos que define los métodos disponibles para ser utilizados por el cliente.

⦿ Estrategia Concreta [Eliminatorias y TodosContraTodos]:

- Implementan las variantes de los algoritmos de las operaciones definidas en la interfaz estrategia: **calcularCantFechas()** y **calcularCantPartidos()**.

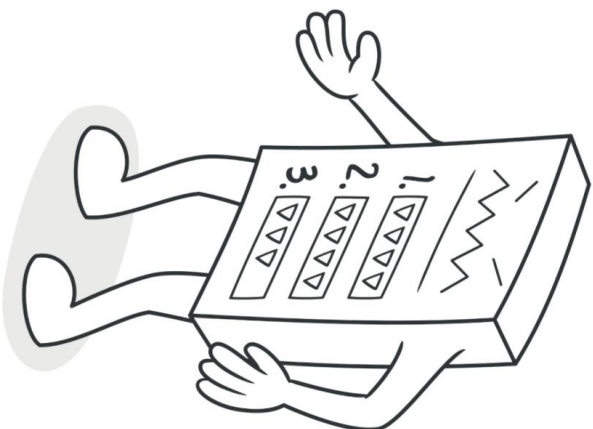
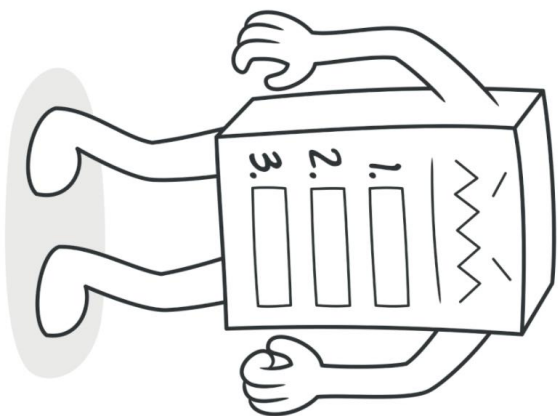
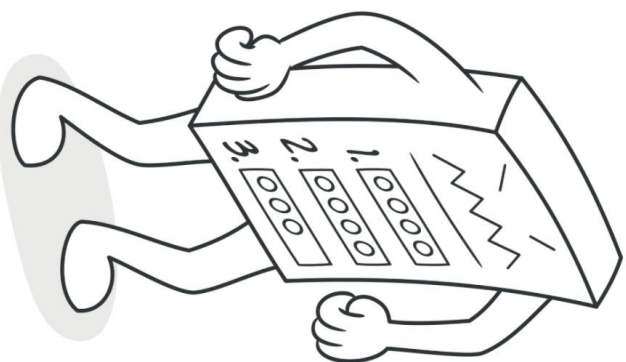
⦿ Contexto [SistemaCampeonato]

- Es la clase que utiliza las diferentes estrategias para ciertas tareas. Mantiene una referencia a una o más instancias Estrategia Concreta, que podrán ser configurables en tiempo de ejecución.
- La configuración puede ser de una o varias estrategias, y debe realizarse antes de invocarse el método de variación algorítmica de cada una de las Estrategias Concretas.
- El Contexto es responsable establecer la estrategia a utilizar y de ejecutar el o los métodos polimórficos. En este caso

State/Strategy

- El Patrón State permite a un objeto tener varios comportamientos diferentes que están basados en su estado interno.
- A diferencia de una máquina de estados procedural, el patrón State representa un estado como una clase completa.
- El “contexto” obtiene su comportamiento delegando al estado actual del objeto por el que está compuesto.
- Encapsulando cada estado en una clase, localizamos cualquier cambio que sea necesario hacer.
- Los patrones STATE/STRATEGY tienen el mismo diagrama de clases, pero con una intención diferente.
- El patrón STRATEGY comúnmente configura la clase contexto con un comportamiento o algoritmo.
- Las transiciones de los estados pueden ser controladas por las clases “estado” o por las clases “contexto”.
- La utilización del patrón STATE resultará en un incremento en el número de clases de su diseño.
- Las clases estado pueden compartirse entre instancias de contexto.

Template Method



Template Method

⦿ Propósito

- ⦿ Define el esqueleto (esquema, patrón) de un algoritmo en una operación, difiriendo algunos pasos a las subclases. Permite a las subclases redefinir ciertos pasos de un algoritmo sin cambiar la estructura del algoritmo.

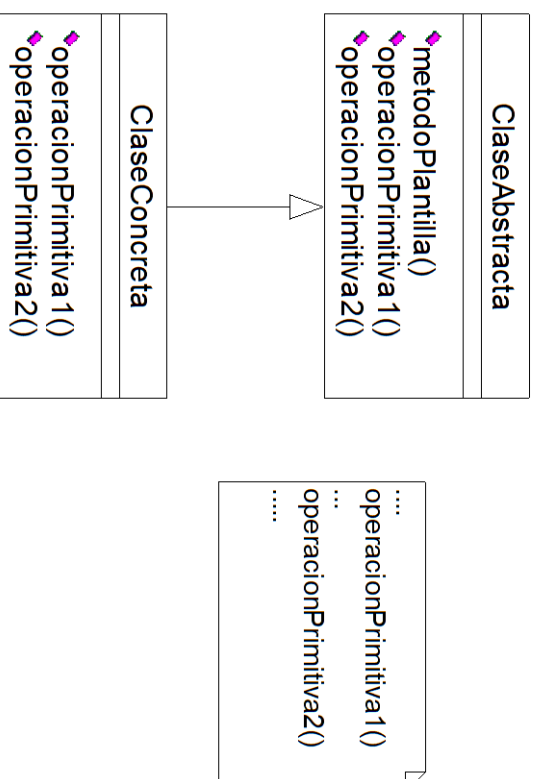
⦿ Motivación

- ⦿ Fundamental para escribir código en un framework.
- ⦿ Clase *Aplicación* que maneja objetos de la clase *Documento*: método *openDocument*

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

Template Method

Estructura



- **Patrones de Creación**
- **Patrones de Estructura**
- **Patrones de Comportamiento**

Template Method:

Aplicación

class Template Method

«control»	
GestorCatalogo	
+	conocerArticulo(art :Articulo) : String[]



«entity»	
Articulo	
-	codigo: String
-	titulo: String
-	productora: Productora[]
-	idiomaOriginal: Idioma[]

+	getCodigo() : String
+	getIdiomaOriginal() : Idioma
+	getTitulo() : String
+	conocerProductora() : String[]
+	conocerIdiomaOriginal() : String[]
+	mostrarInfoProducto() : String[]
+	mostrarInfoEspecific() : String[]
+	conocerTraducciones() : String[]



«entity»	
Productora	
-	nombre: String
+	getNombre() : String



«entity»	
Idioma	
-	nombre: String
+	getNombre() : String



«entity»	
Pelicula	
-	calificacion: Calificacion
-	rubro: Rubro
-	idiomaSubtitulo: Idioma
+	getCalificacion() : Calificacion
+	getIdiomaSubtitulo() : Idioma[]
+	mostrarInfoEspecific() : String[]
+	conocerTraducciones() : String[]



«entity»	
Calificacion	
-	nombre: String
+	getNombre() : String



«entity»	
Rubro	
-	nombre: String
+	getNombre() : String

- El Template Method (método plantilla) es el método "mostrarInfoProducto()".
- Los métodos "mostrarInfoEspecific()" y "conocerTraducciones()" son los métodos abstractos variables que cada subclase concreta implementará de manera diferente.
- Los métodos "getTitulo()", "conocerProductora()" y "conocerIdiomaOriginal()" son los métodos de enganche (hook).

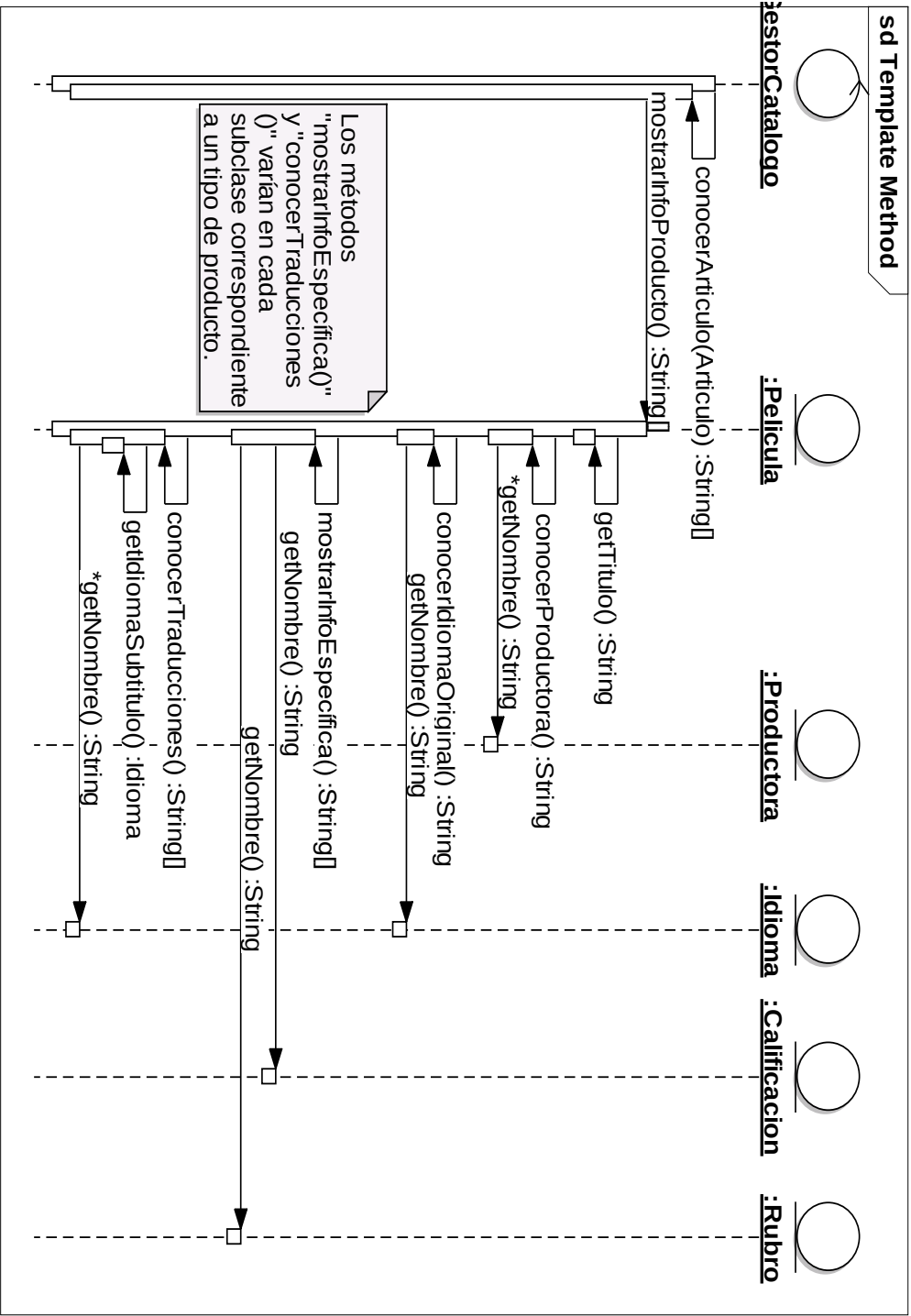
Estructura

Template

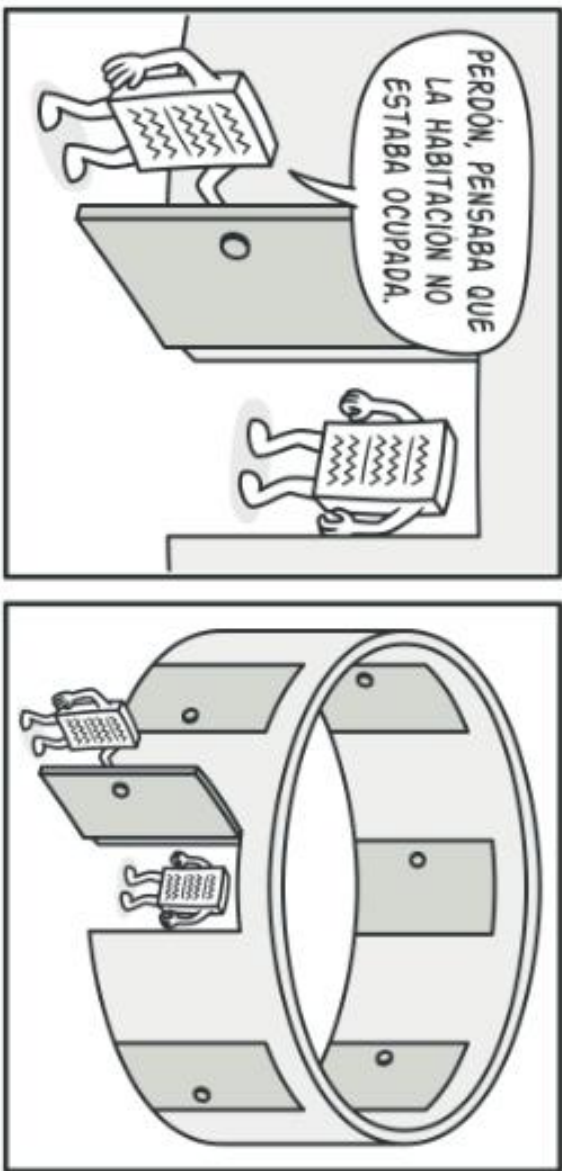
Method:

Aplicación

Dinámica



Singleton



Singleton

Propósito

- Asegurar que una clase tiene una única instancia y asegurar un acceso global

Motivación

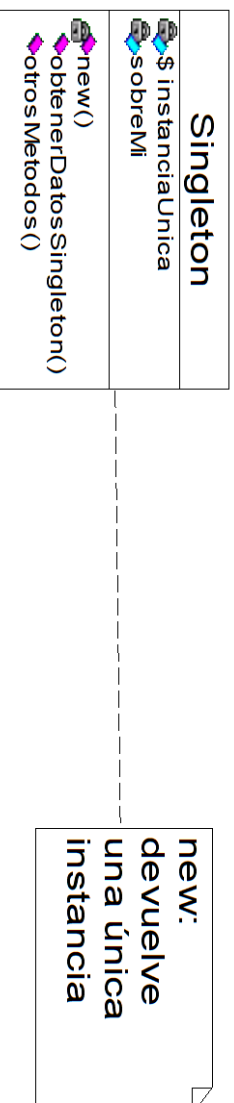
- La clase se encarga de asegurar que exista una única instancia y de su acceso.
- Hay muchas circunstancias en las cuales es necesario esto:
 - Caches
 - Cajas de Diálogo
 - Objetos que manejan preferencias y configuraciones de registros.
 - Objetos utilizados para logueo
 - Objetos manejadores de dispositivos como impresoras o tarjetas gráficas.
- Para estos objetos, instanciar más de uno provocaría toda clase de problemas como: funcionamiento incorrecto, sobreuso de recursos, resultados inconsistentes.

Patrones de Creación

- Patrones de Estructura
- Patrones de Comportamiento

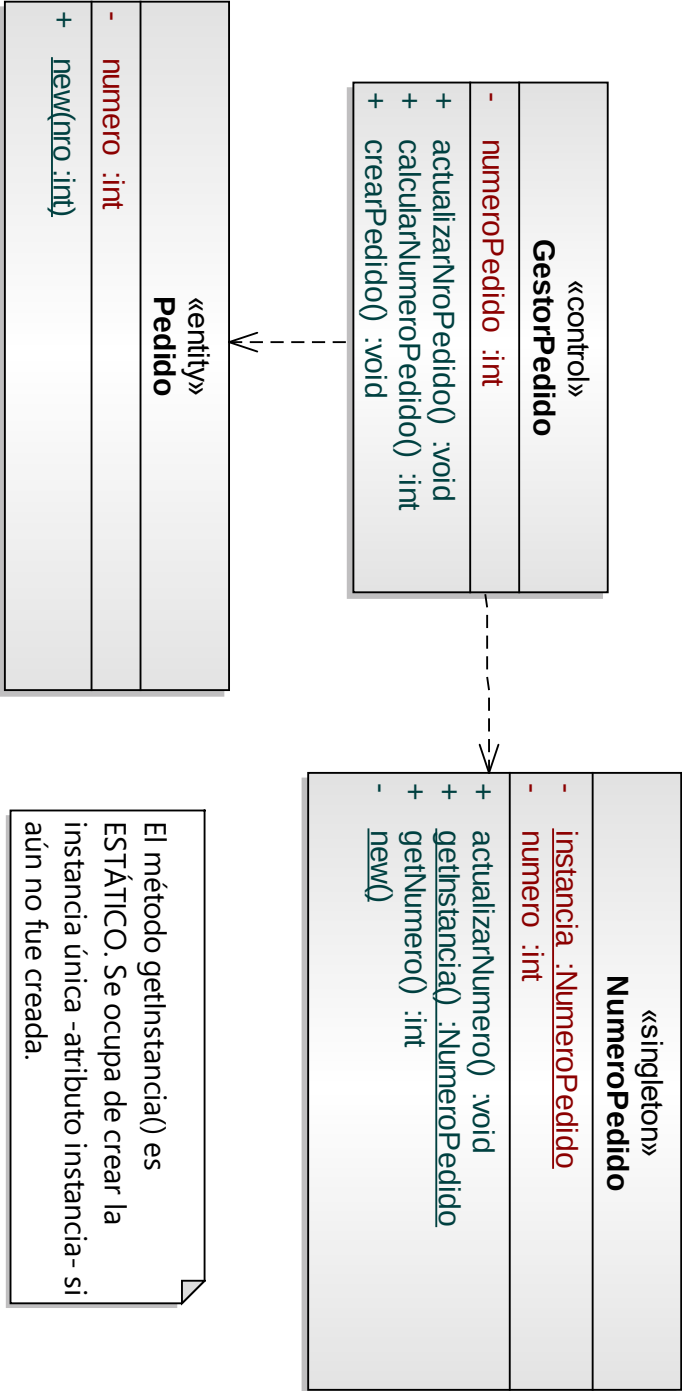
Singleton

Estructura



Singleton: Aplicación

Estructura



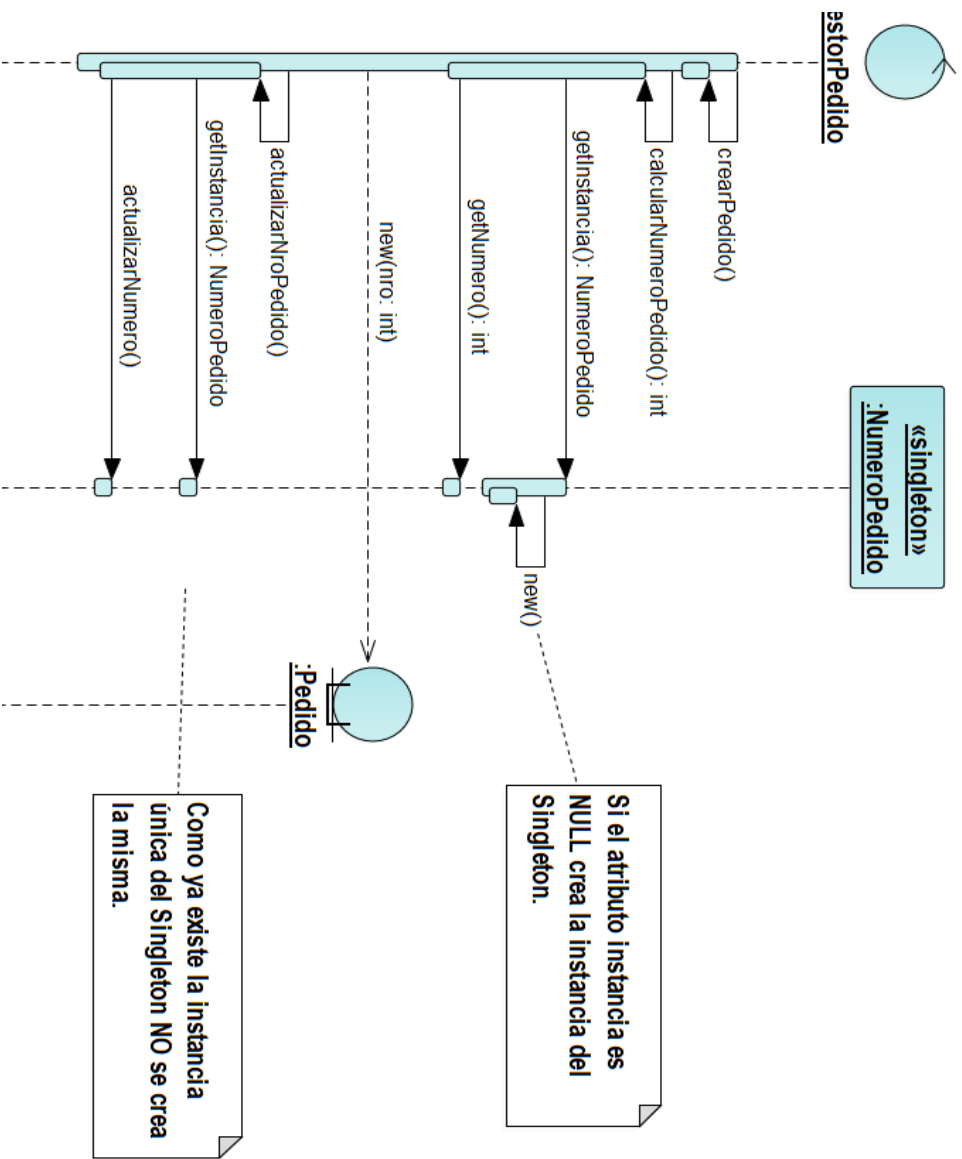
Singleton:

Aplicación

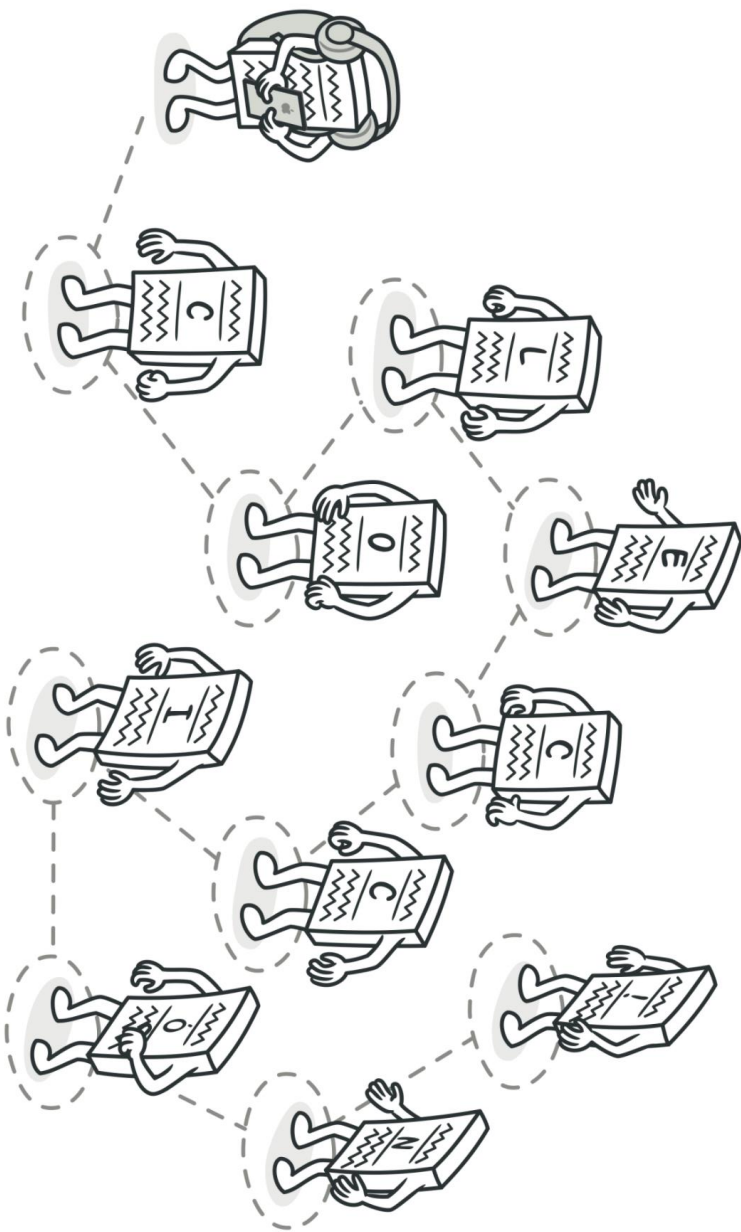
⦿ Dinámica

⦿ Ventaja

Permite un acceso controlado a la única instancia ya que posee un control estricto sobre cómo y cuándo acceden a ella.



Iterador (Iterator)



Iterator

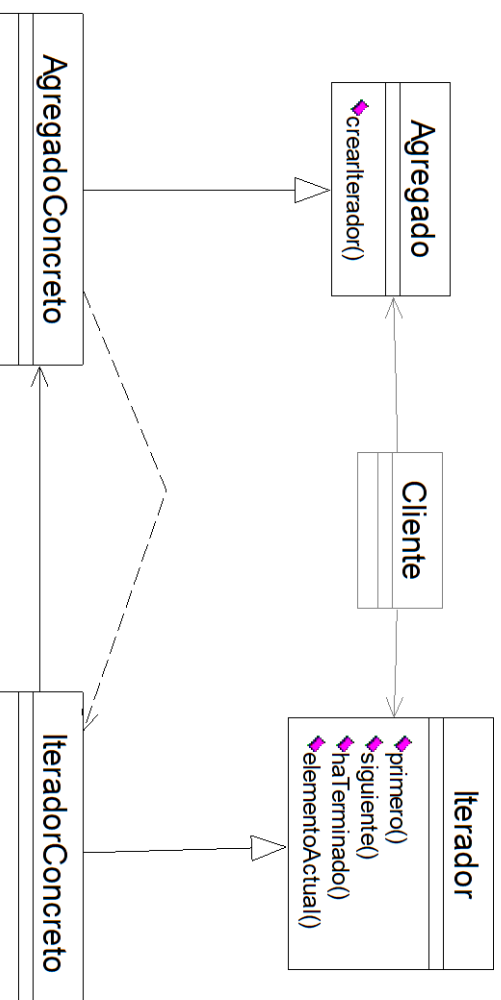
Consecuencias

- Simplifica la interfaz de un contenedor al extraer los métodos de recorrido
- Permite varios recorridos, concurrentes
- Soporta variantes en las técnicas de recorrido

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

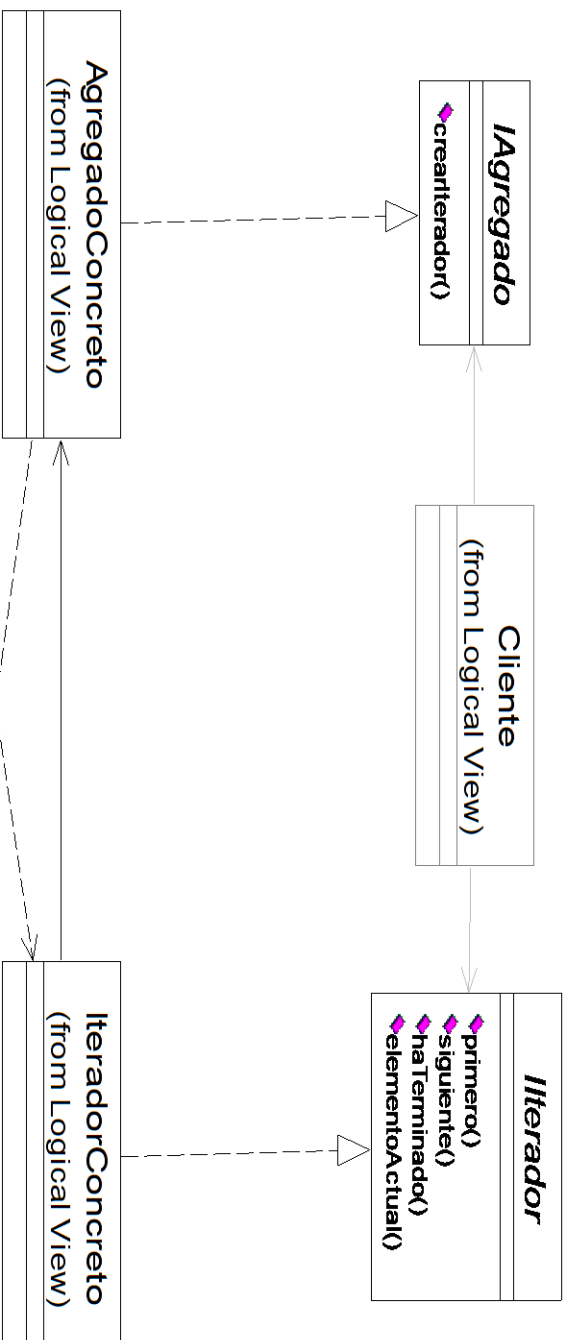
Iterador Externo Polimórfico

Herencia



Iterador Externo Polimórfico

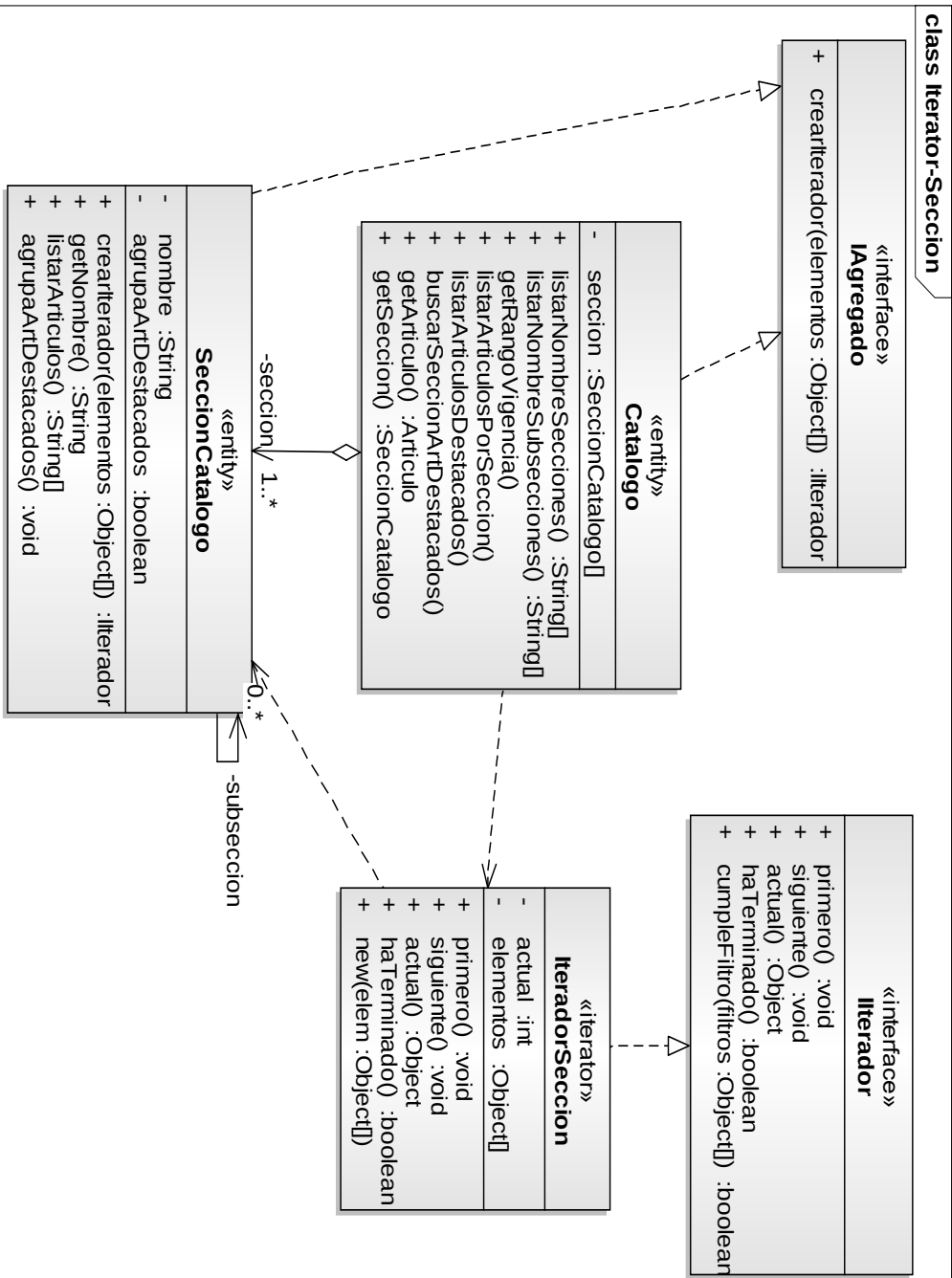
Realización

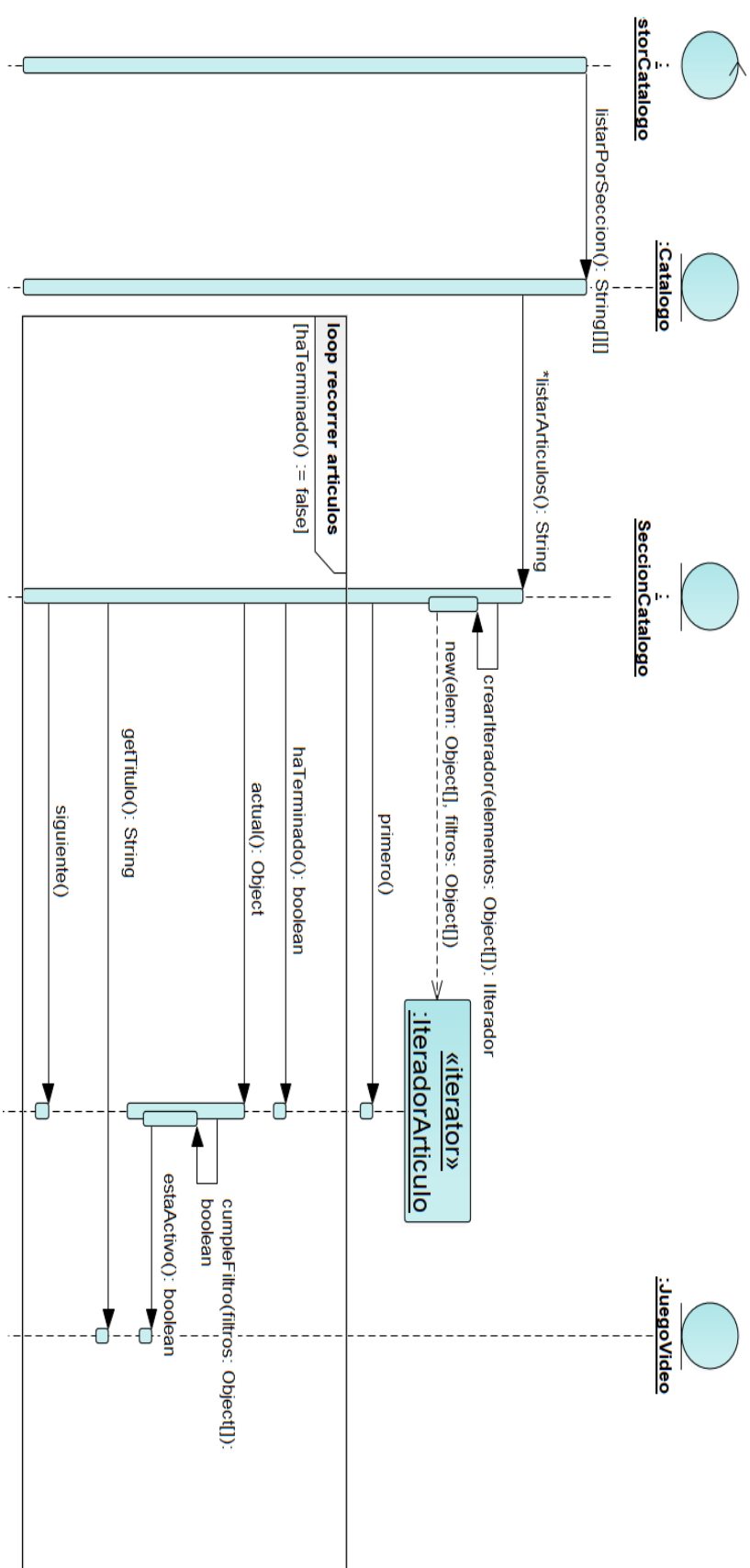


Iterator:

Aplicación

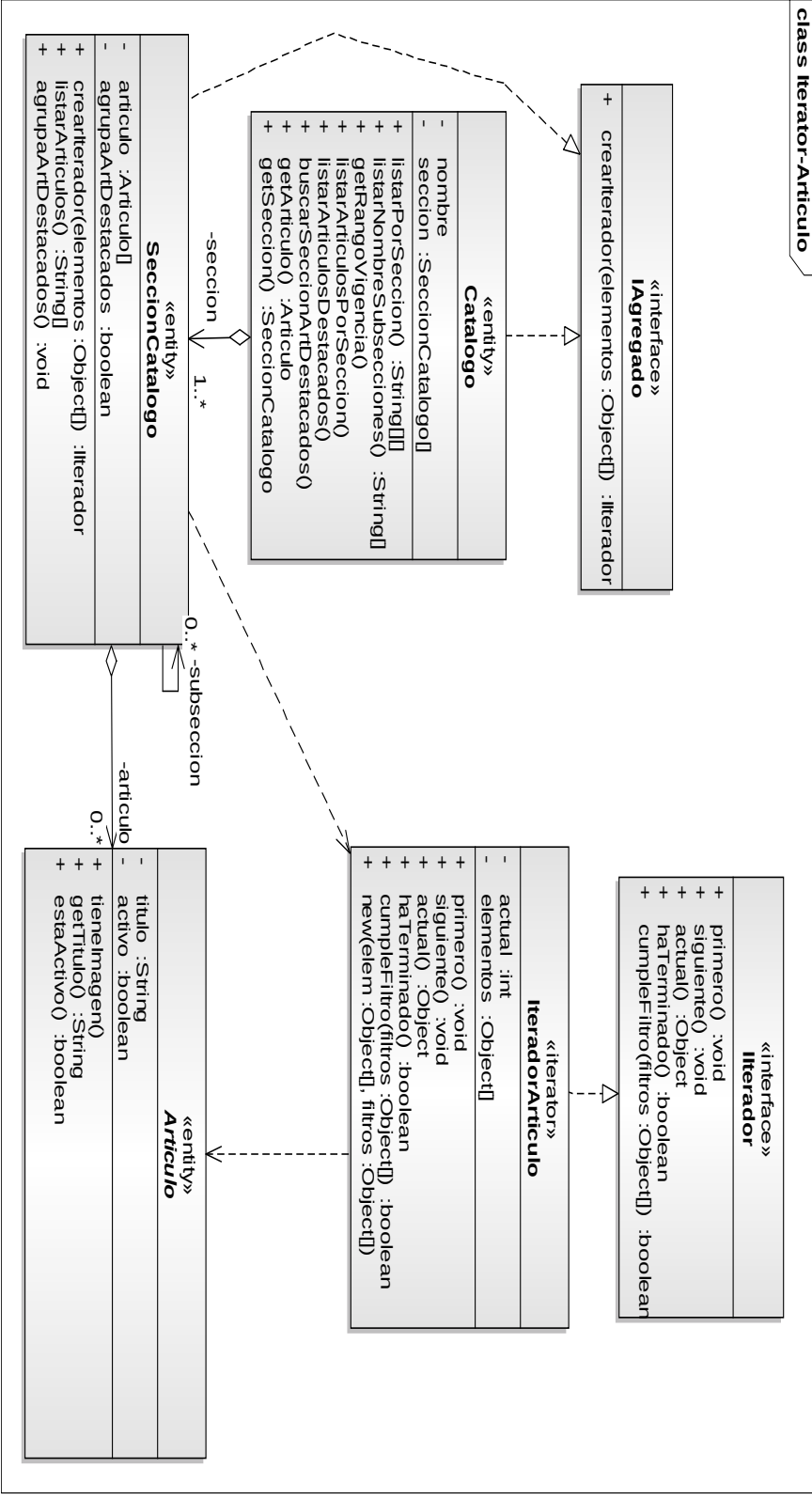
Estructura





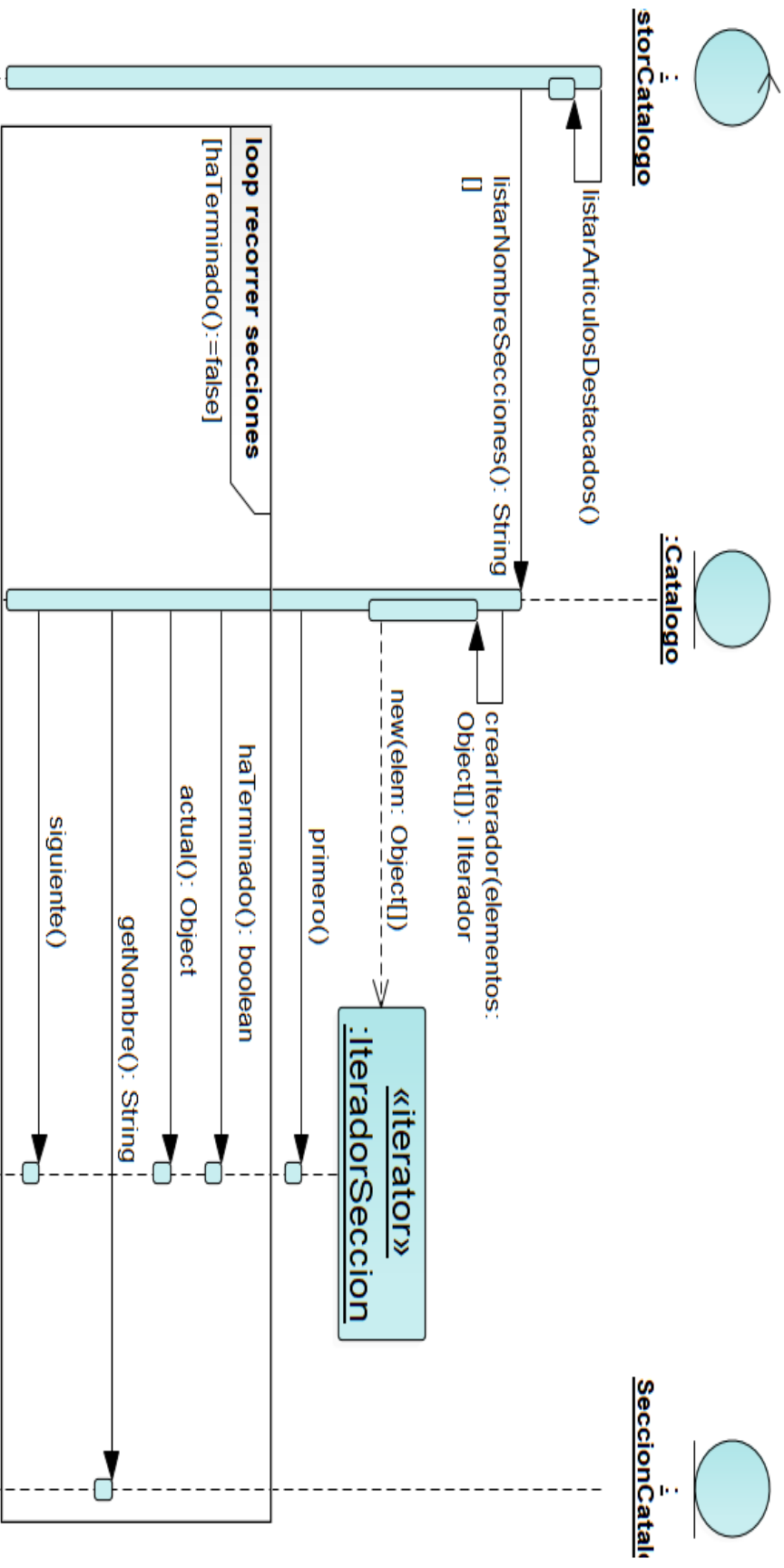
Iterator: Aplicación

Estructura

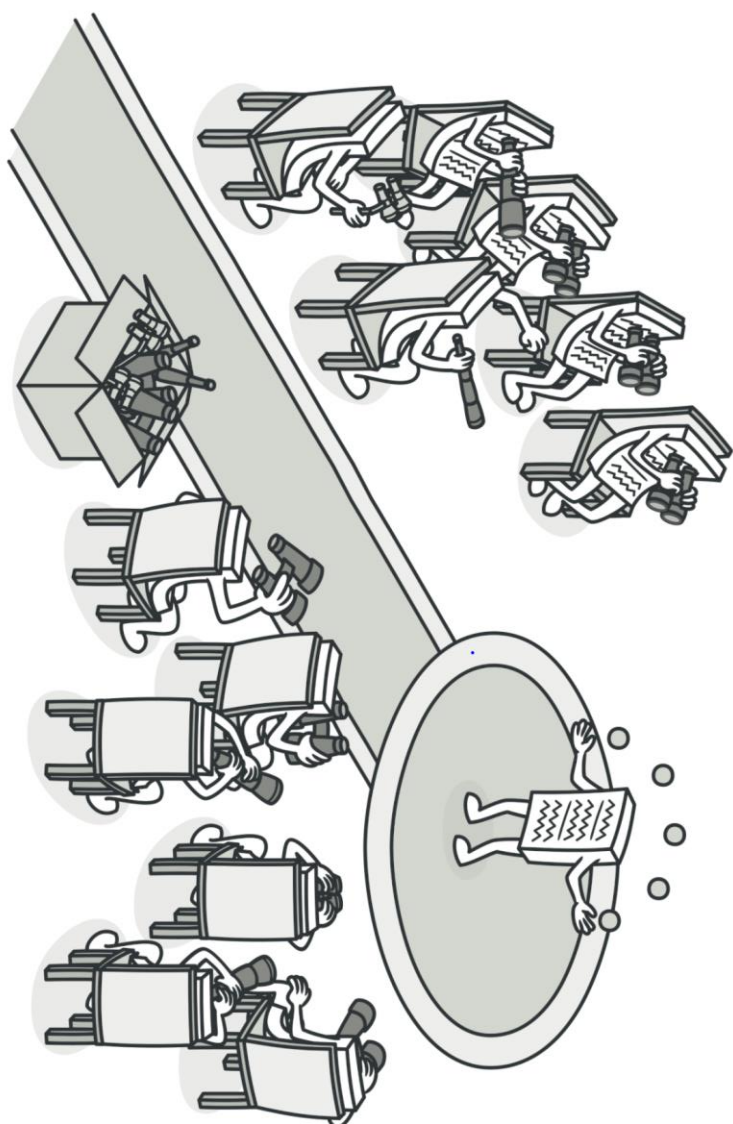


Dinámica:

Iterator: Aplicación



Observer



Observer / Publish-Subscribe

◎ Propósito

- Define una dependencia uno-a-muchos entre objetos, de modo que cuando cambia el estado de un objeto, todos sus dependientes automáticamente son notificados y actualizados.

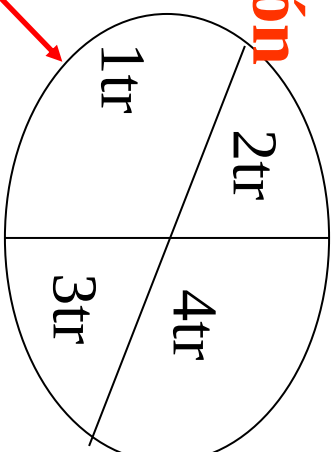
◎ Motivación

- ¿Cómo mantener la consistencia entre objetos relacionados, sin establecer un acoplamiento fuerte entre sus clases?

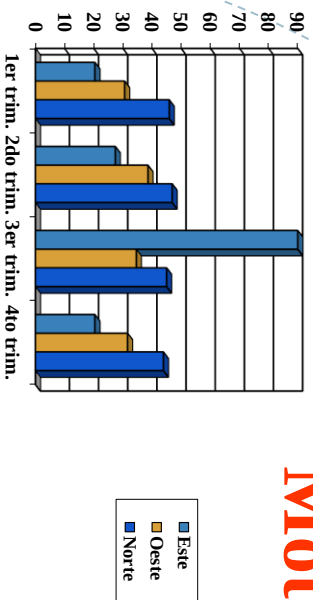
- Ejemplo: Separación *Modelo-Vista*

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

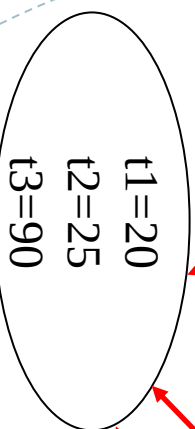
Motivación



Vistas



Modelo

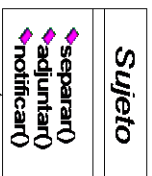


	<u>1tr.</u>	<u>2tr.</u>	<u>3tr.</u>	<u>4tr.</u>
Este	20	25	90	20
Oeste	30	38	32	32
Norte	47	47	45	45

Observer

Estructura

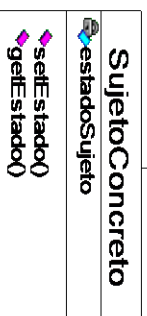
Para todos los O en
observadores {O ->
update() }



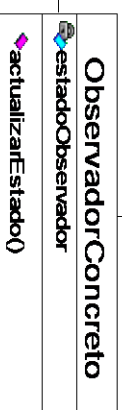
1



1..*



+theSujetoConcreto

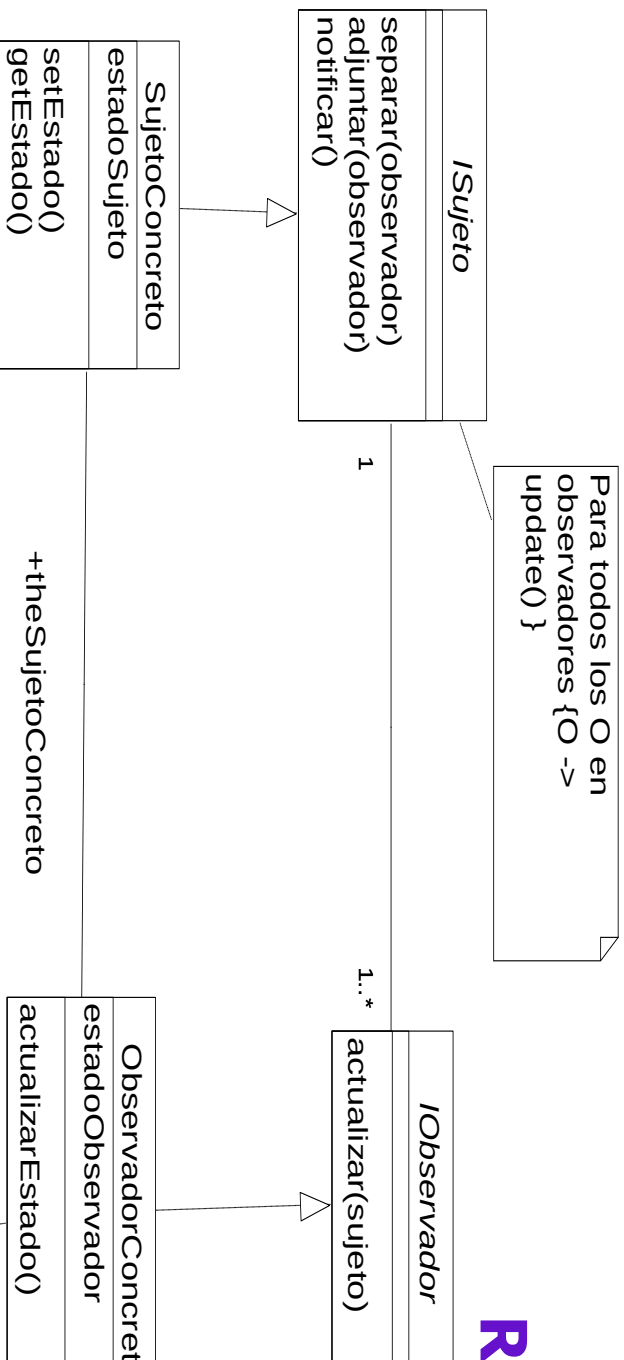


estadoObservador= Sujeto
-> getEstado()

Herencia

Observer

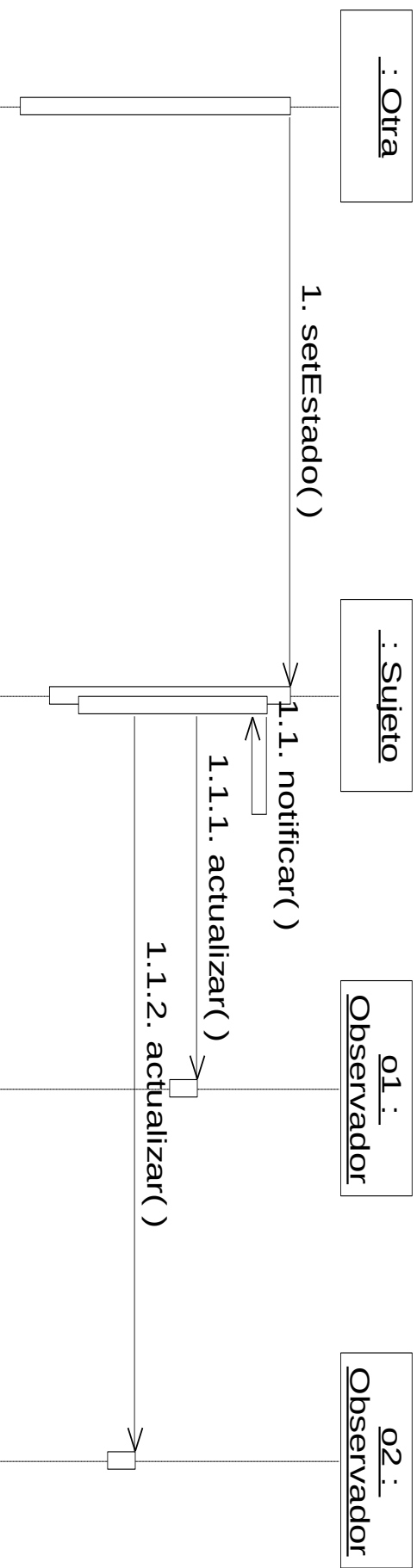
Estructura



Realización

estadoObservador = Sujeto
-> getEstado()

Colaboración



Observer

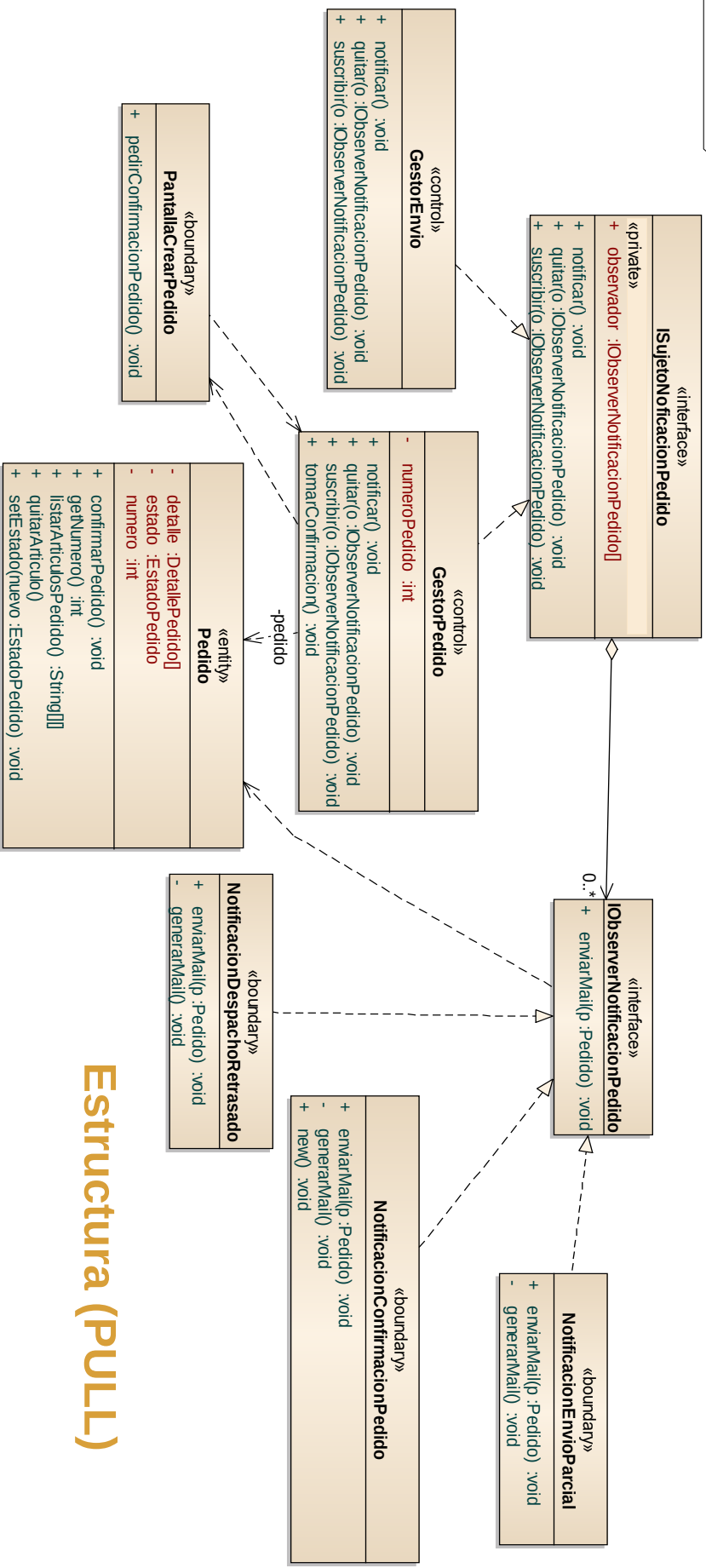
⦿ Consecuencias

- ⦿ Soporte para comunicación de tipo *broadcast*:
 - el *subject* no especifica al receptor concreto de la notificación
 - es posible añadir y eliminar *observers* en cualquier instante.
- ⦿ Actualizaciones inesperadas
- ⦿ Interfaz de *Observer* simple requiere que los *observers* tengan que deducir el ítem cambiado

- Patrones de Creación
- Patrones de Estructura
- Patrones de Comportamiento

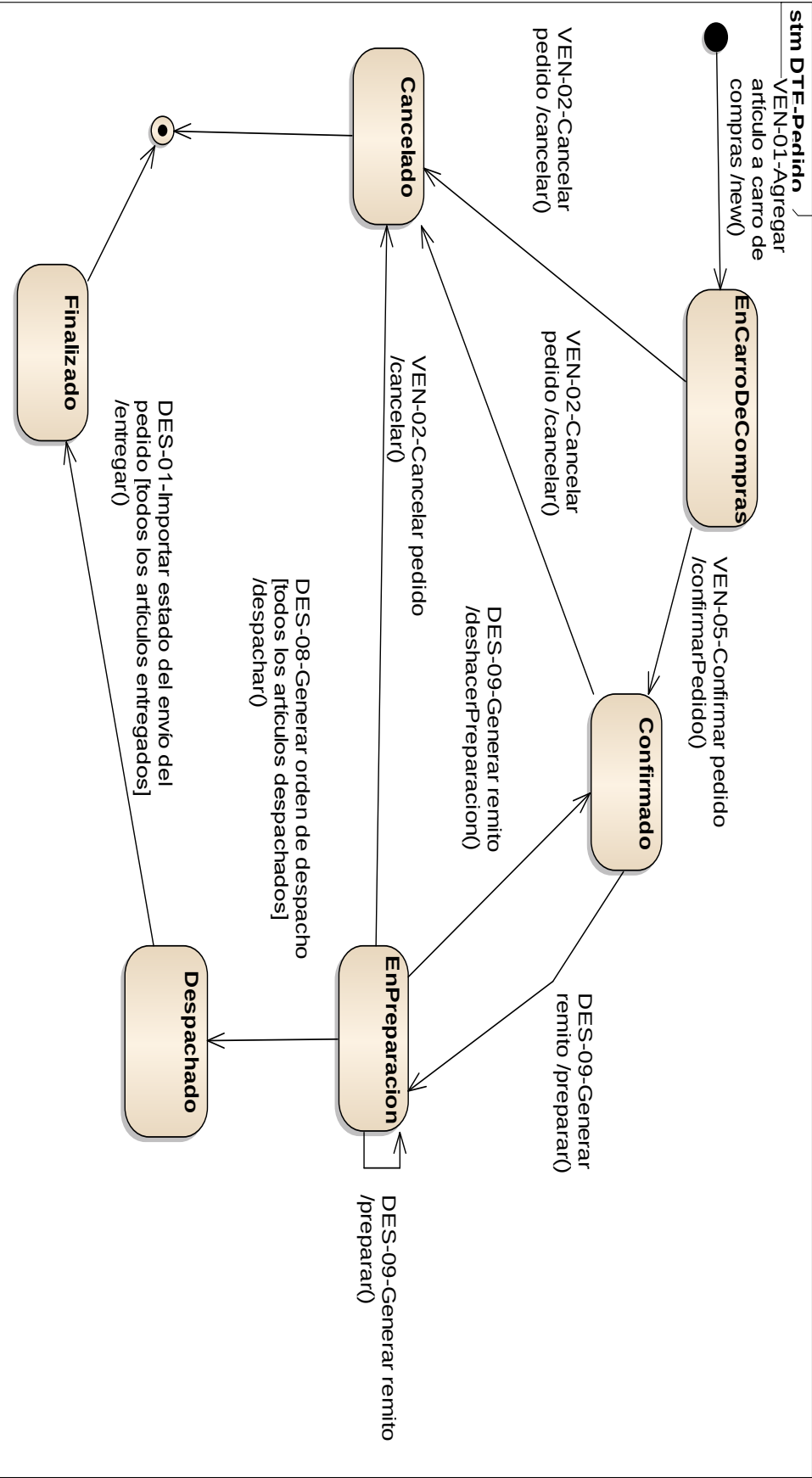
Observer: Aplicación

class Observer-Pull



Estructura (PULL)

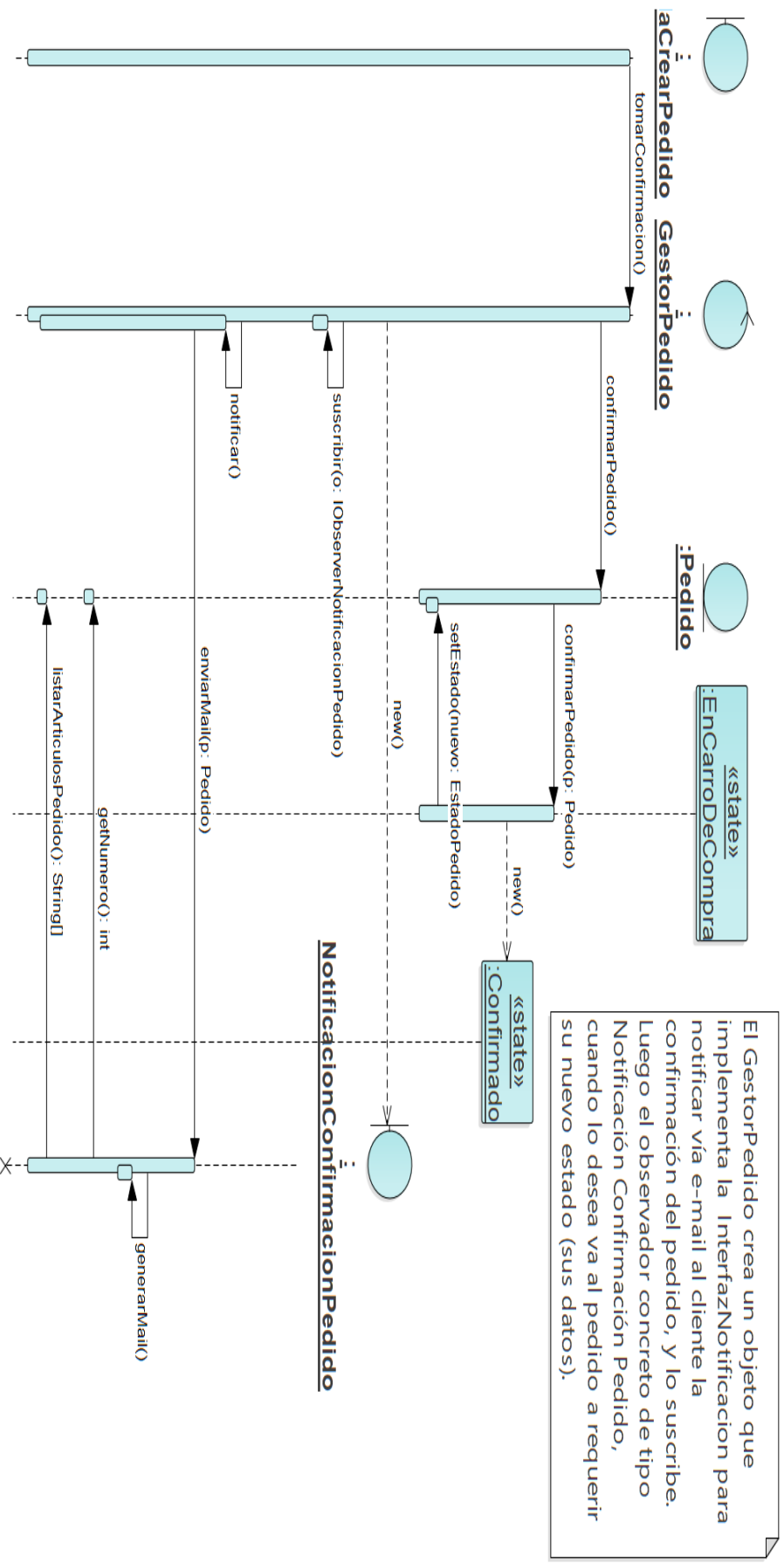
Máquina de Estado de la Clase Pedido



Observer : Aplicación

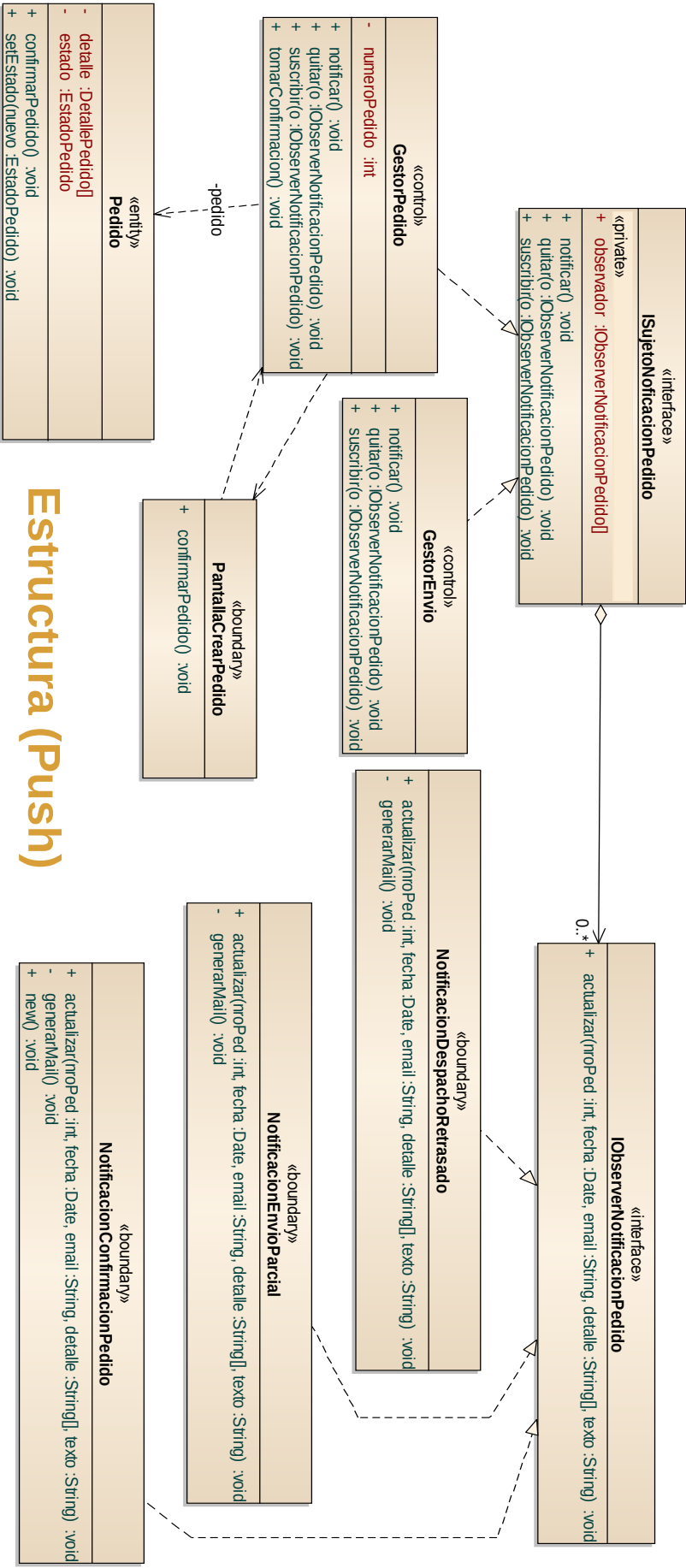
Dinámica (Pull)

sd Observer-Pull



Observer: Aplicación

class Observer-Push

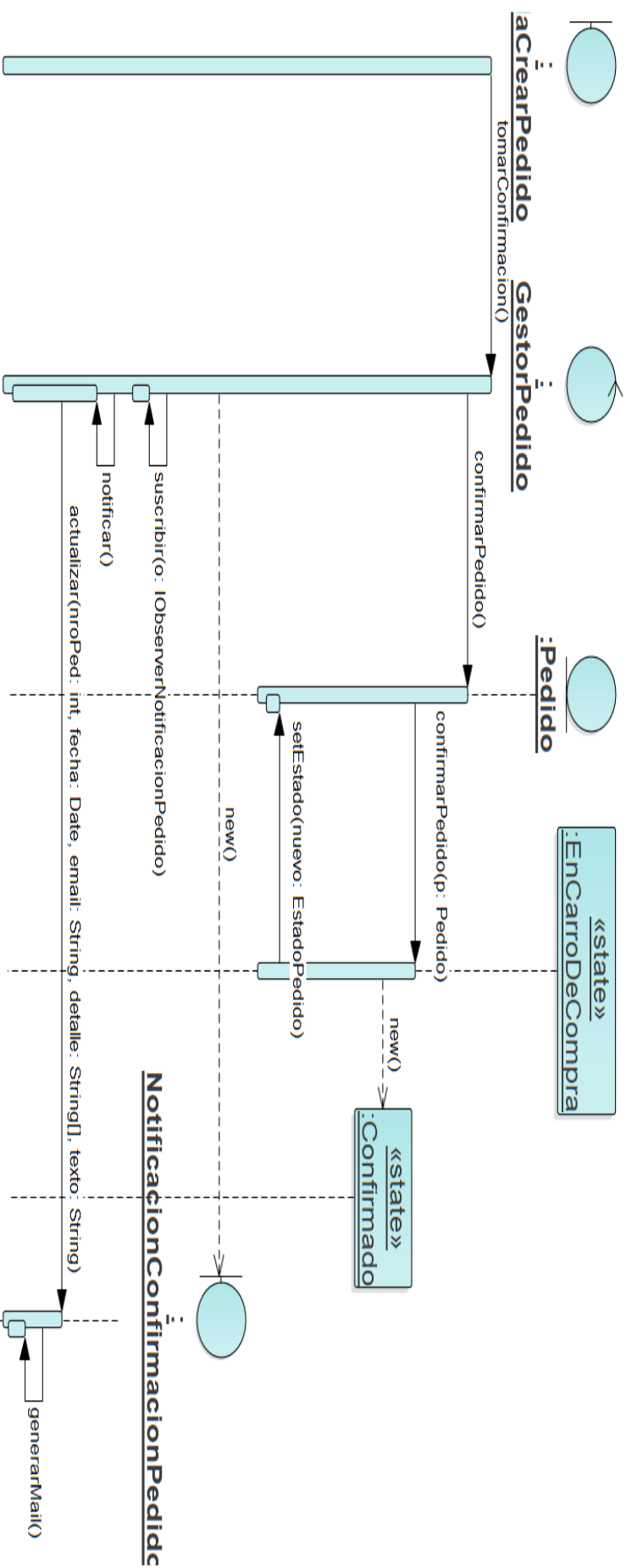


Estructura (Push)

Observer : Aplicación

Dinámica (Push)

Observer-Push



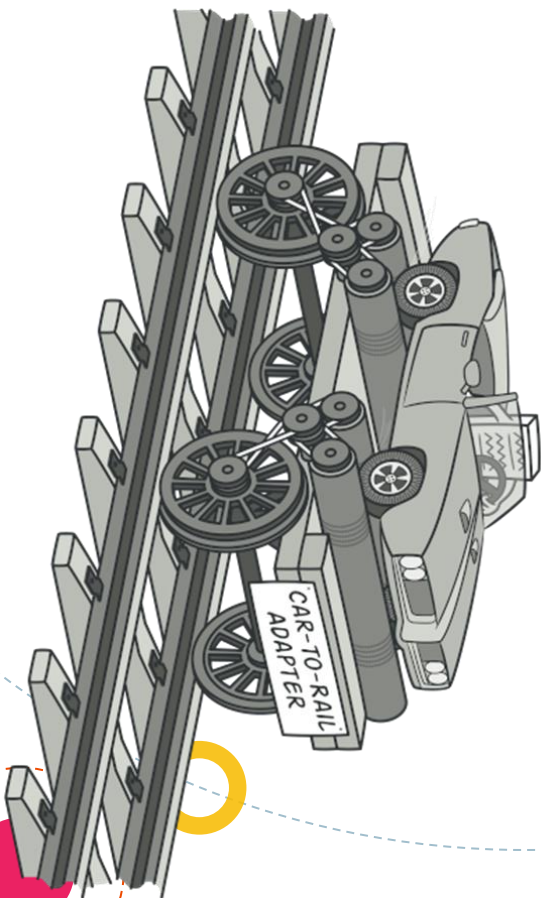
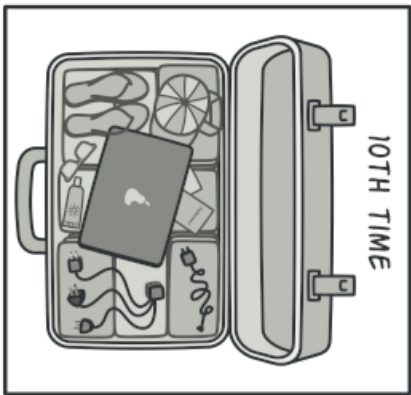
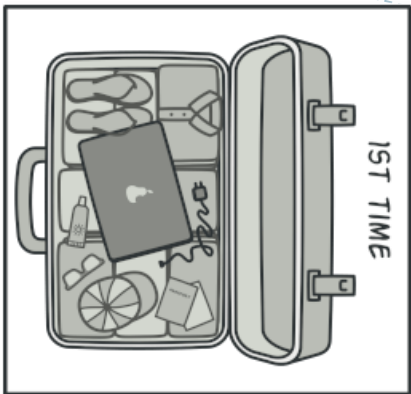
NotificacionConfirmacionPedido

Tras producirse el cambio de estado del Pedido, se notifica y se envía datos forzosamente a los observadores suscritos al sujeto GestorPedido. Luego los Observadores disponen de esos datos para realizar sus operaciones.

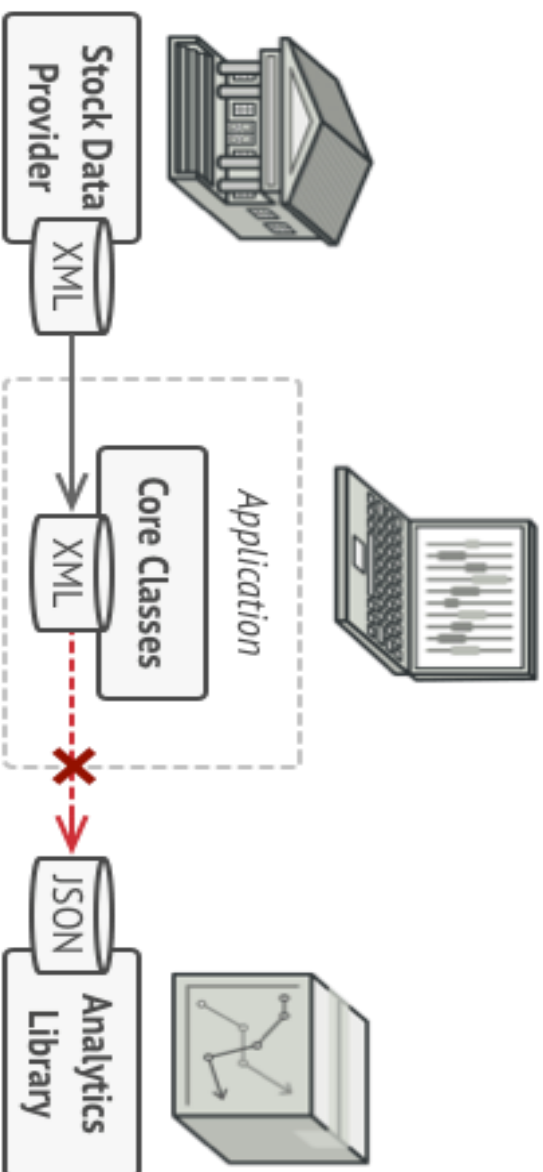
Adapter / Wrapper

(Adaptador / Envoltorio)

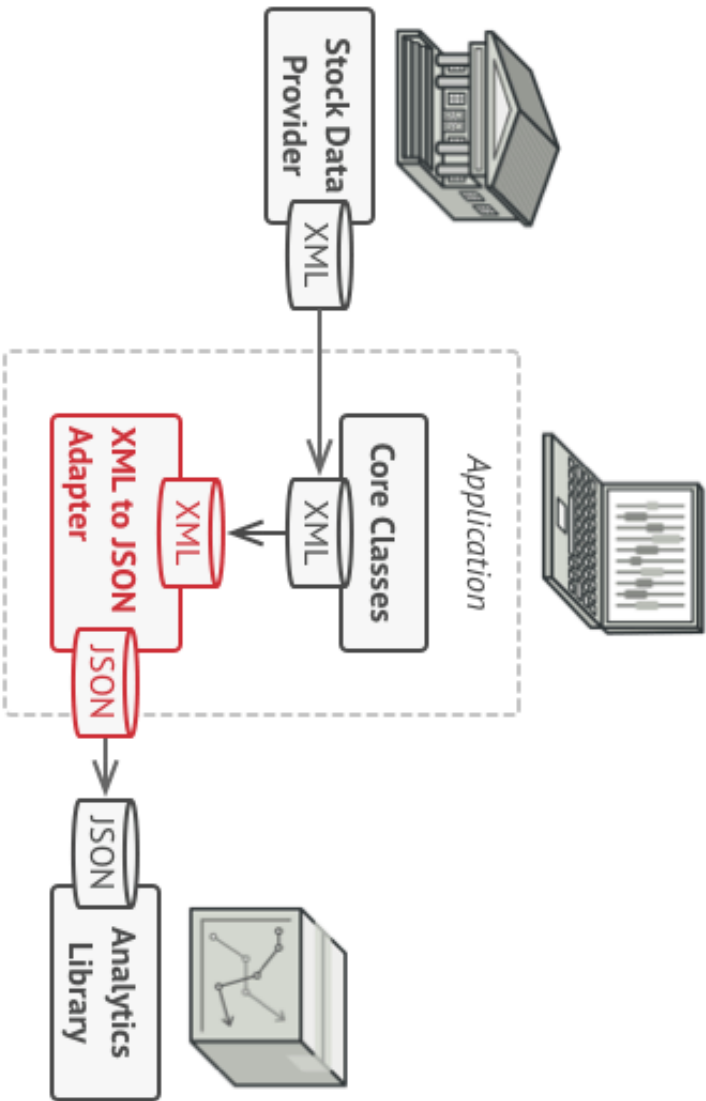
TRAVELING ABROAD



¿Qué problema resuelve?



Solución: Patrón Adaptador



Solución: Patrón Adaptador

Propósito

Convertir la interfaz de un objeto para que otro pueda comprenderla.

Se dice que el Adaptador “**envuelve**” uno de los objetos para ocultar la complejidad de la conversión. El objeto “envuelto” no necesita conocer que se realiza una conversión.

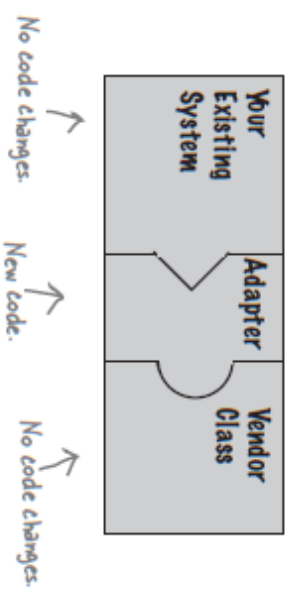
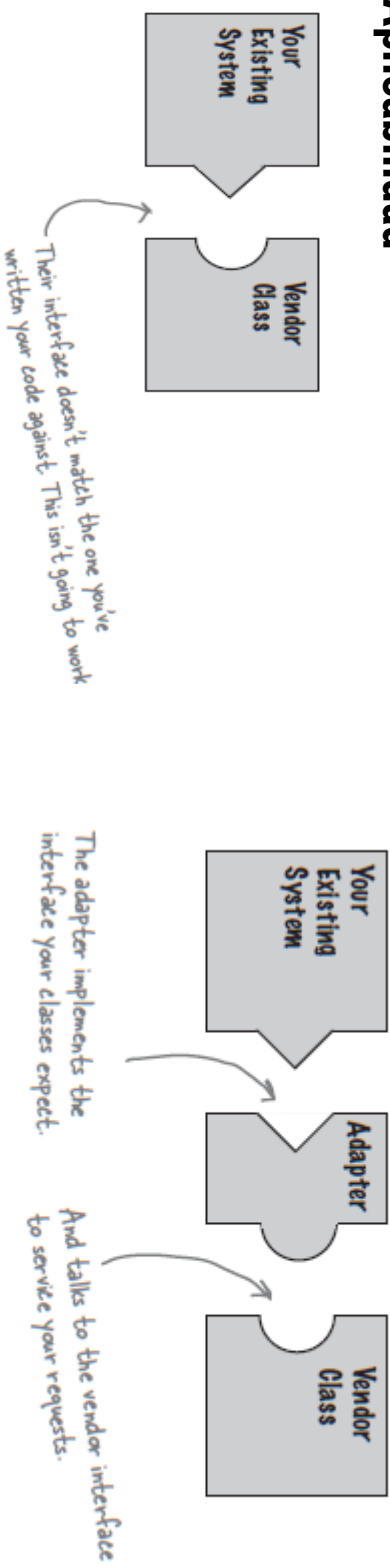
Ámbito

De clases y de objetos



Solución: Patrón Adaptador

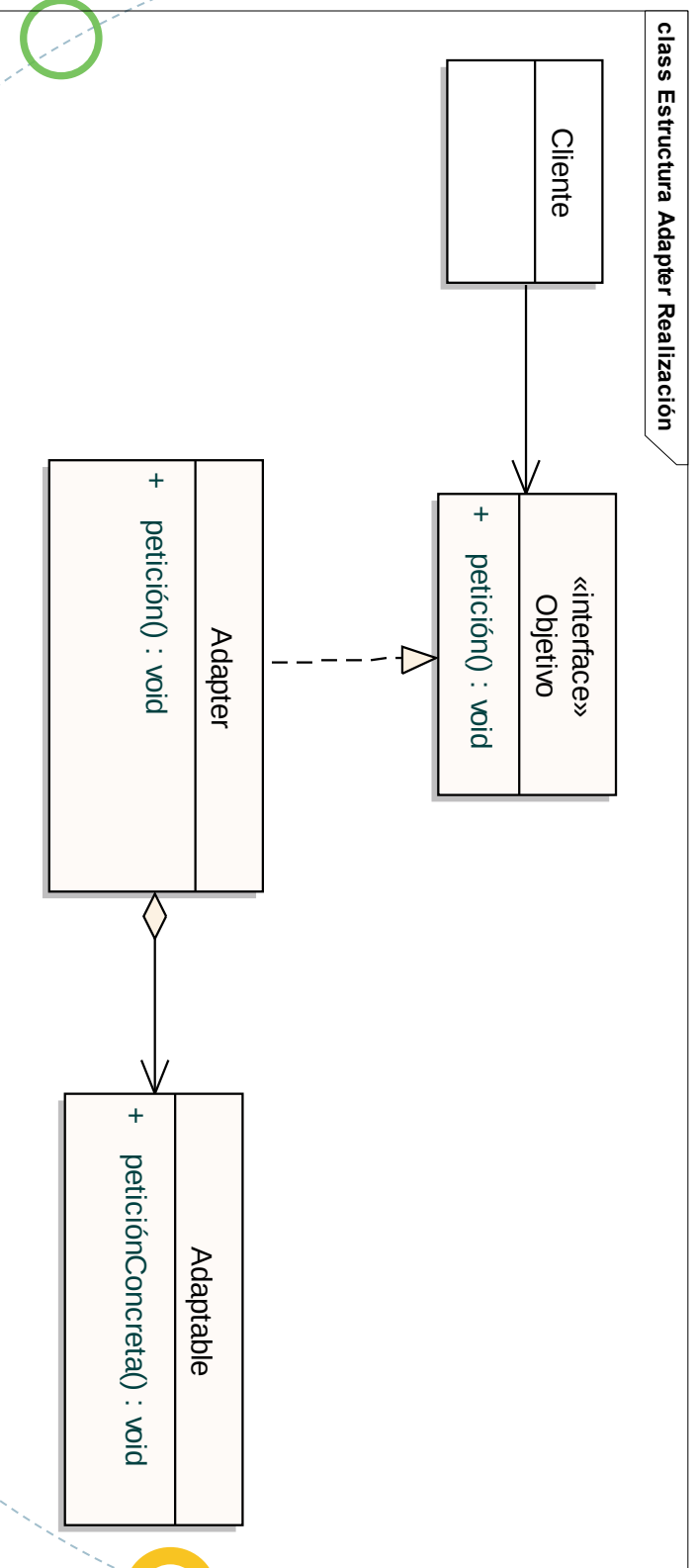
Aplicabilidad



Estructura – Adaptador de Objeto

Adapter 123

Esta implementación utiliza el principio de composición de objetos: el adaptador implementa la interfaz de un objeto y envuelve el otro. Se puede implementar en todos los lenguajes de programación populares.

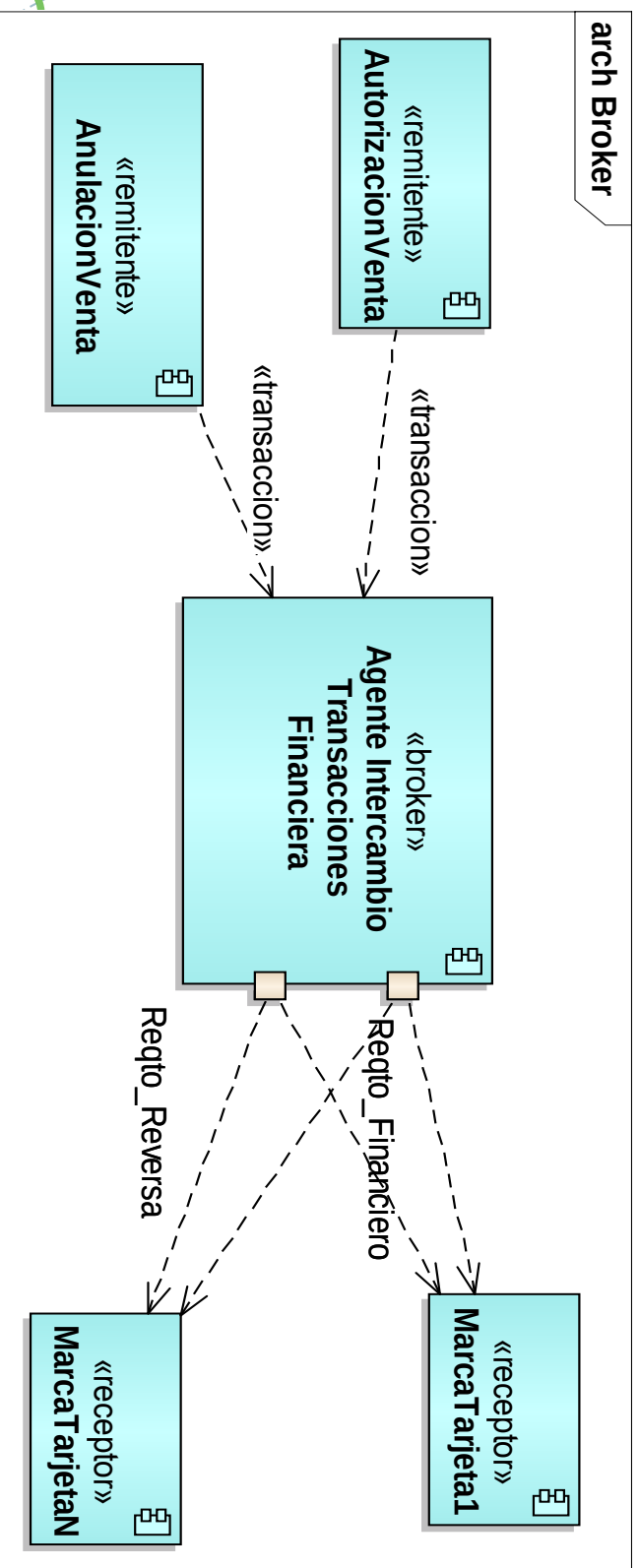


Aplicación Venta Digital

Implementar la integración con los sistemas de autorizaciones de pago propio de cada marca de tarjeta de crédito. Esta integración implica gestionar los mensajes de requerimientos de solicitud financiera (venta) y de reversa de operación (anulación de venta) respetando el estándar de transacciones financiera ISO 8583.

Aplicación Venta Digital

¿Y si miramos la arquitectura?



Aplicación Venta Digital

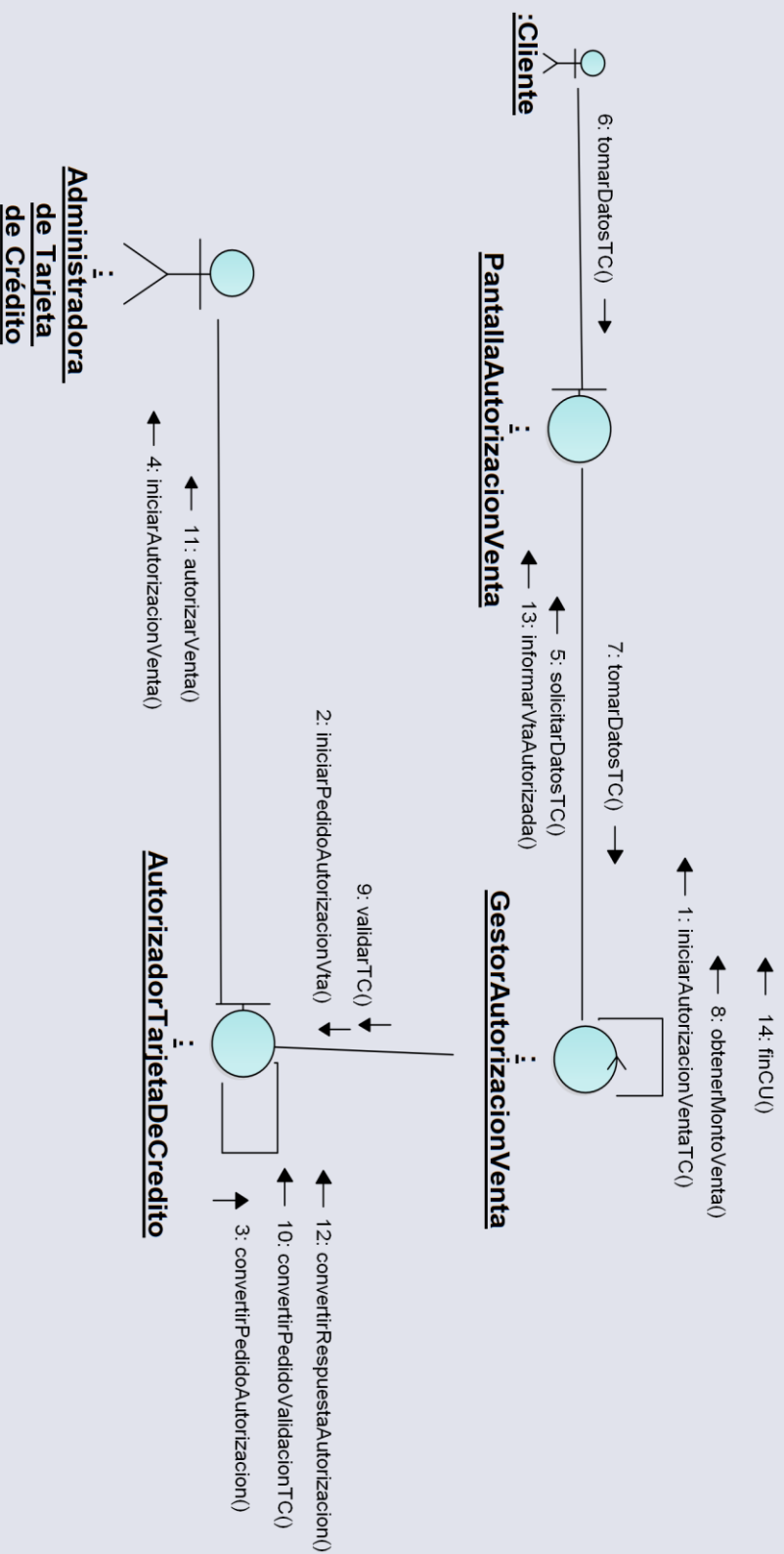
Estructura Análisis – CU Autorizar Venta con Tarjeta de Crédito



Aplicación Venta Digital

Adapater 127

Dinámica Análisis – CU Autorizar Venta con Tarjeta de Crédito



Aplicación Venta Digital

Estructura Diseño – CU Autorizar Venta con Tarjeta de Crédito

«control»	
GestorAutorizacionVenta	
-	codigoVerificadorTC: int
-	fechaVtoTC: Date
-	montoVenta: float
-	nroAutorizacionViaTC: int
-	nroTarjetaCredito: char
+	finCU(): void
+	iniciarAutorizacionVentaTC(): void
+	obtenerMontoVenta(): float
+	tomarDatosTC(): int

«interface»	
AutorizadorTarjetaDeCredito	
+	convertirPedidoAutorizacion(numeroTarjeta: char, fechaVto: char, control: int, montoVenta: double): void
+	convertirRespuestaAutorizacion(): void
+	validarTC(numeroTarjeta: char, fechaVto: char, control: int, montoVenta: double, cantidadCuotas: int): int

«adapter» AutorizadorTarjetaDeCreditoMastercard

- + convertirPedidoAutorizacion(numeroTarjeta: char, fechaVto: char, control: int, montoVenta: double): void
- + convertirRespuestaAutorizacion(): void
- + validarTC(numeroTarjeta: char, fechaVto: char, control: int, montoVenta: double, cantidadCuotas: int): int

«adapter» AutorizadorTarjetaDeCreditoVisa

- + convertirPedidoValidacionTC(numeroTarjeta: char, fechaVto: Date, control: int, montoVenta: float): void
- + convertirRespuestaAutorizacion(): void
- + validarTC(numeroTarjeta: char, fechaVto: Date, control: int, montoVenta: double, cantidadCuotas: int): int

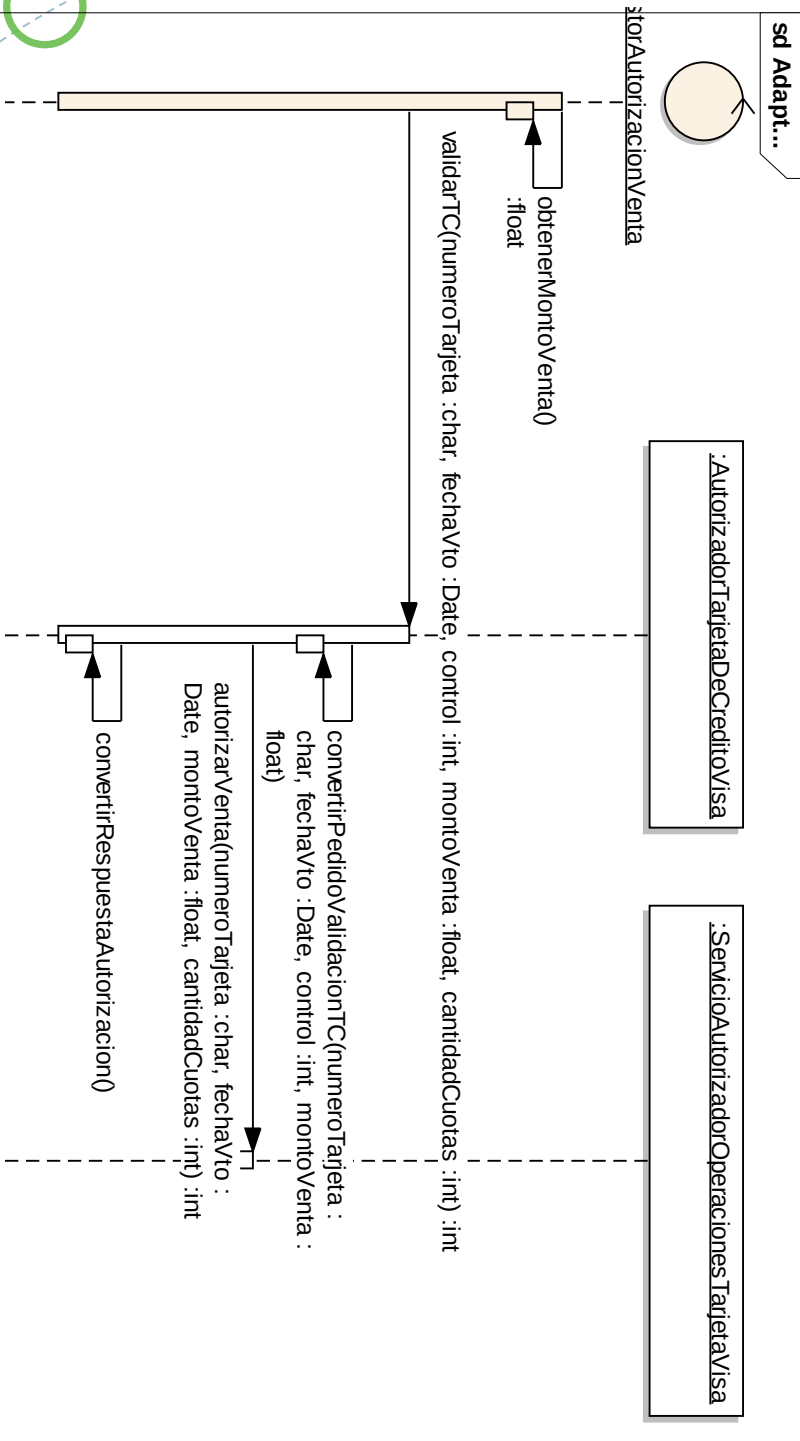
ServicioAutorizacionTarjetaMastercard

- + autorizarVentaMastercard(numeroTarjeta: char, fechaVto: char, montoVenta: double, cantidadCuotas: int): int

ServicioAutorizadorOperacionesTarjetaVisa

- + autorizarVenta(numeroTarjeta: char, fechaVto: Date, montoVenta: float, cantidadCuotas: int): int

Dinámica Diseño – CU Autorizar Venta con Tarjeta de Crédito



Estructura – Adaptador de Clase

Esta implementación utiliza la herencia: el adaptador hereda las interfaces de ambos objetos al mismo tiempo. Tenga en cuenta que este enfoque solo se puede implementar en lenguajes de programación que admitan herencia múltiple.

